A project report on

NEUROIMAGING ROBOT POWERED BY BRAIN FOUNDATION MODELS

Submitted in partial fulfillment for the award of the degree of

Bachelor of Technology in Computer Science and Engineering

by

SRINJAY SARKAR (23BRS1007)
SUJATRA BISWAS (23BAI1478)
MAHARGHA KUNDU (23BCE5009)



VIT®
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)
CHENNAI

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

October, 2026

NEUROIMAGING ROBOT POWERED BY BRAIN FOUNDATION MODELS

Submitted in partial fulfillment for the award of the degree of

Bachelor of Technology in Computer Science and Engineering

by

SRINJAY SARKAR (23BRS1007)
SUJATRA BISWAS (23BAI1478)
MAHARGHA KUNDU (23BCE5009)



VIT®
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)
CHENNAI

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

October, 2026

VIT®
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)
CHENNAI

DECLARATION

I hereby declare that the thesis entitled "NEUROIMAGING ROBOT POWERED BY BRAIN FOUNDATION MODELS" submitted by SRINJAY SARKAR (23BRS1007), for the award of the degree of Bachelor of Technology in Computer Science and Engineering, Vellore Institute of Technology, Chennai is a record of bonafide work carried out by me under the supervision of Dr. Shridevi S.

I further declare that the work reported in this thesis has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Place: Chennai

Date: **Signature of the Candidate**

VIT®
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)
CHENNAI

School of Computer Science and Engineering

CERTIFICATE

This is to certify that the report entitled "NEUROIMAGING ROBOT POWERED BY BRAIN FOUNDATION MODELS" is prepared and submitted by Srinjay Sarkar (23BRS1007) to Vellore Institute of Technology, Chennai, in partial fulfillment of the requirement for the award of the degree of Bachelor of Technology in Computer Science and Engineering is a bonafide record carried out under my guidance. The project fulfills the requirements as per the regulations of this University and in my opinion meets the necessary standards for submission. The contents of this report have not been submitted and will not be submitted either in part or in full, for the award of any other degree or diploma and the same is certified.

Signature of the Guide:

Name: Dr. Shridevi S

Date:

Signature of the Examiner

Name:

Date:

Signature of the Examiner

Name:

Date:

Approved by the Head of Department,
B.Tech. Computer Science and Engineering

Name:
Date:

(Seal of the School of Computer Science and Engineering)

ABSTRACT

Brain MRI gives a three-dimensional view of the brain, but building a reliable classifier from a small labelled dataset is hard. In this project we adapted BrainIAC (Brain Imaging Adaptive Core), a pretrained brain MRI foundation model, to sort preprocessed NIfTI volumes into four classes: Normal, Mild Cognitive Impairment (MCI), Alzheimer's disease and Stroke. The system combines a BrainIAC-based 3D Vision Transformer Base (3D ViT-B) backbone, a four-class classification head, evaluation scripts, ONNX export and a Gradio interface for inference and slice viewing. The dataset has 60 volumes, 15 per class. In the first recorded split, with 42 scans for training, 9 for validation and 9 for testing, the model reached 44.44% accuracy, a macro-F1 of 0.4464 and a macro one-vs-rest ROC-AUC of 0.7579. It identified both Stroke test scans and missed all three Alzheimer's scans. A later Jetson package reports a separate five-fold, subject-grouped cross-validation with 78.33% accuracy, 0.7784 macro-F1 and 0.9248 macro-AUC (reported 95% confidence interval 0.8753 to 0.9678). The two sets of numbers come from different evaluation records and are not results of a single experiment. The package also contains a shortcut check suggesting that part of the high score may come from differences in data source or acquisition. The Gradio interface supports scan upload, predicted labels with score bars, processing times and axial, coronal and sagittal slice views. Running the full application on an NVIDIA Jetson with a touchscreen is the next step. The current system is an engineering research prototype, not a clinically validated diagnostic tool.

Keywords: Brain MRI, BrainIAC, foundation model, 3D Vision Transformer, medical image classification, ONNX, edge AI, NVIDIA Jetson, NIfTI

ACKNOWLEDGEMENT

It is my pleasure to express with a deep sense of gratitude to Dr. Shridevi S, Professor and Deputy Director, Centre for Neuroinformatics, Vellore Institute of Technology, Chennai, for her constant guidance, continual encouragement and understanding throughout this project. Working with her went beyond academics. It gave me the chance to learn from an expert in neuroinformatics and to see how brain imaging foundation models can be adapted for an embedded AI application.

It is with gratitude that I would like to extend my thanks to the visionary leader Dr. G. Viswanathan, our Honorable Chancellor, Mr. Sankar Viswanathan, Dr. Sekar Viswanathan and Dr. G. V. Selvam, Vice Presidents, Dr. Sandhya Pentareddy, Executive Director, Ms. Kadhambari S. Viswanathan, Assistant Vice-President, Dr. V. S. Kanchana Bhaaskaran, Vice-Chancellor, Dr. T. Thyagarajan, Pro-Vice Chancellor, VIT Chennai, and Dr. P. K. Manoharan, Additional Registrar, for providing an exceptional working environment and inspiring all of us during the tenure of the course.

Special mention to Dr. Viswanathan V, Dean, Dr. Nithyanandam P, Associate Dean, Dr. Suganya G, Associate Dean, and Dr. Sweetlin Hemalatha C, Associate Dean, School of Computer Science and Engineering, Vellore Institute of Technology, Chennai, for spending their valuable time and effort in sharing their knowledge and for helping us in every aspect.

I express my wholehearted thanks to the Head of the Department, B.Tech. Computer Science and Engineering, and to the Project Coordinators for their support and encouragement in taking up and completing this work. I also thank my team members, Mahargha Kundu and Sujatra Biswas, for their cooperation in preparing the data, fine-tuning the model, analysing the results and building the interface.

My sincere thanks to all the faculty and staff at Vellore Institute of Technology, Chennai, who helped me acquire the knowledge this project needed. I would also like to thank the BrainIAC authors, whose published work made this study possible, my parents for their support, and my friends who encouraged me to take up and complete this task.

Place: Chennai

Date: **Srinjay Sarkar (23BRS1007)**

CONTENTS

ABSTRACT ..... **i**
ACKNOWLEDGEMENT ..... **ii**
LIST OF TABLES ..... **vi**
LIST OF FIGURES ..... **vii**
LIST OF ACRONYMS ..... **viii**

CHAPTER 1
INTRODUCTION **1**
1.1 BACKGROUND AND CONTEXT 1
1.2 MOTIVATION 1
1.3 PROBLEM STATEMENT 2
1.4 RESEARCH QUESTIONS 2
1.5 OBJECTIVES 3
1.6 SCOPE AND BOUNDARIES 3
1.7 EXPECTED CONTRIBUTIONS 4
1.8 ORGANISATION OF THE REPORT 4

CHAPTER 2
LITERATURE REVIEW **5**
2.1 BRAIN MRI AS VOLUMETRIC DATA 5
2.2 CONVENTIONAL DEEP LEARNING FOR MRI 5
2.3 FOUNDATION MODELS AND SELF-SUPERVISED LEARNING 6
2.4 BRAINIAC: GENERAL-PURPOSE BRAIN MRI FOUNDATION MODEL 6
2.5 VISION TRANSFORMERS AND CONTRASTIVE LEARNING 7
2.6 RELATED NEUROIMAGING FOUNDATION MODELS 7
2.7 PREPROCESSING AND DOMAIN SHIFT 8
2.8 EDGE AI AND MODEL INTERCHANGE 8
2.9 RESEARCH GAP AND PROJECT POSITIONING 9
2.10 COMPARISON OF SELECTED MODEL PARADIGMS 9
2.11 SUMMARY 9

CHAPTER 3
METHODOLOGY AND SYSTEM DESIGN **11**
3.1 SYSTEM OVERVIEW 11
3.2 DATASET CURATION AND LABELS 12
3.3 INPUT PREPARATION 13
3.4 MODEL ARCHITECTURE 13
3.5 FINE-TUNING PROCEDURE 14
3.6 DATA SPLITS AND VALIDATION 14

3.7 EVALUATION METRICS 15
3.8 ONNX EXPORT AND INFERENCE CONTRACT 16
3.9 GUI DESIGN 17
3.10 TARGET JETSON ARCHITECTURE 17
3.11 END-TO-END ALGORITHM 17
3.12 DESIGN RATIONALE AND RISKS 18

CHAPTER 4
IMPLEMENTATION AND EXPERIMENTAL SETUP 19
4.1 DATASET AND FILE ORGANISATION 19
4.2 FINE-TUNING WORKFLOW 19
4.3 EVALUATION ARTIFACTS 19
4.4 ONNX PACKAGE AND INFERENCE 20
4.5 GUI IMPLEMENTATION 21
4.6 DEPLOYMENT PLAN AND CURRENT STATUS 21
4.7 REPRODUCIBILITY AND QUALITY CHECKS 22
4.8 ETHICAL AND SAFETY CONSIDERATIONS 22
4.9 SUMMARY 23

CHAPTER 5
RESULTS AND ANALYSIS 24
5.1 DATASET AND SPLIT SUMMARY 24
5.2 TRAINING AND VALIDATION BEHAVIOUR 24
5.3 INITIAL HELD-OUT TEST RESULTS 25
5.4 CONFUSION MATRIX AND PER-CLASS BEHAVIOUR 26
5.5 ROC AND PRECISION-RECALL ANALYSIS 28
5.6 CROSS-VALIDATION RESULTS IN THE JETSON PACKAGE 29
5.7 RESULT RECONCILIATION AND LABEL-ORDER RISK 30
5.8 GUI DEMONSTRATION 31
5.9 TIMING AND INFERENCE CONSIDERATIONS 33
5.10 LIMITATIONS AND THREATS TO VALIDITY 33
5.11 SUMMARY 34

CHAPTER 6
CONCLUSION AND FUTURE WORK 35
6.1 SUMMARY OF WORK 35
6.2 KEY FINDINGS 35
6.3 LIMITATIONS 36
6.4 FUTURE WORK 36
6.5 FINAL CONCLUSION 37

REFERENCES 39

APPENDICES **41**
APPENDIX 1: DATASET MANIFEST 41
APPENDIX 2: MODEL INPUT/OUTPUT CONTRACT 43
APPENDIX 3: INFERENCE TEST PROCEDURE 44
APPENDIX 4: METRIC DEFINITIONS 46
APPENDIX 5: JETSON AND TOUCHSCREEN CHECKLIST 47
APPENDIX 6: ARTIFACT INVENTORY 48
APPENDIX 7: FULL TRAINING HISTORY 49
APPENDIX 8: FINAL REPORTING CHECKLIST 50

LIST OF TABLES

2.1	COMPARISON OF SELECTED BRAIN-IMAGING MODEL PARADIGMS	9
3.1	CLASS-WISE DATASET INVENTORY	12
3.2	EVALUATION METRICS AND LIMITATIONS	15
3.3	MODEL INPUT/OUTPUT CONTRACT RECORDED IN THE JETSON PACKAGE	16
4.1	ARTIFACT AND SOFTWARE INVENTORY	20
4.2	CURRENT IMPLEMENTATION STATUS AND COMPLETION CRITERIA	21
5.1	CLASS COUNTS AND RECORDED SPLIT	24
5.2	INITIAL HELD-OUT TEST METRICS	25
5.3	CONFUSION MATRIX COUNTS BY ACTUAL AND PREDICTED CLASS	26
5.4	INITIAL TEST CLASSIFICATION REPORT BY CLASS	27
5.5	PER-CLASS ONE-VS-REST AUC AND AVERAGE PRECISION	29
5.6	CROSS-VALIDATION METRICS REPORTED BY JETSON PACKAGE METADATA	29
5.7	PER-CLASS METRICS FROM PACKAGE-REPORTED CROSS-VALIDATION	30
6.1	IMPLEMENTATION STATUS AGAINST PROJECT OBJECTIVES	35
A1.1	SCAN MANIFEST FOR ALZHEIMER'S CLASS	41
A1.2	SCAN MANIFEST FOR MCI CLASS	41
A1.3	SCAN MANIFEST FOR NORMAL CLASS	42
A1.4	SCAN MANIFEST FOR STROKE CLASS	42
A2.1	INPUT AND OUTPUT TENSOR SPECIFICATION	43
A3.1	DEVICE VALIDATION RECORD SHEET	45
A4.1	METRIC DEFINITIONS	46
A5.1	JETSON AND TOUCHSCREEN VALIDATION CHECKLIST	47
A6.1	DELIVERABLE ARTIFACT INVENTORY	48
A7.1	TRAINING AND VALIDATION HISTORY BY EPOCH	49

LIST OF FIGURES

3.1 OVERALL SYSTEM ARCHITECTURE AND IMPLEMENTATION STATUS 11

3.2 END-TO-END PROJECT WORKFLOW 11

3.3 CLASS DISTRIBUTION OF THE PREPROCESSED SCAN BUNDLE 12

3.4 ONE RECORDED TRAINING/VALIDATION/TEST SPLIT FROM THE INITIAL EXPERIMENT 15

5.1 TRAINING AND VALIDATION LOSS OVER TEN EPOCHS (INITIAL RUN) 25

5.2 TRAINING AND VALIDATION ACCURACY OVER TEN EPOCHS (INITIAL RUN) 25

5.3 INITIAL HELD-OUT TEST CONFUSION MATRIX; ROW TOTALS ARE 3, 2, 2 AND 2 27

5.4 INITIAL HELD-OUT PER-CLASS PRECISION, RECALL AND F1-SCORE 27

5.5 INITIAL HELD-OUT ONE-VS-REST ROC CURVES AND CLASS-WISE AUC VALUES 28

5.6 INITIAL HELD-OUT ONE-VS-REST PRECISION-RECALL CURVES AND AVERAGE PRECISION VALUES 29

5.7 GUI EXAMPLE: NORMAL SCAN OUTPUT FROM THE TEMPORARY INTERFACE 31

5.8 GUI EXAMPLE: MCI PREDICTION FOR A SCAN WITH KNOWN MCI LABEL 32

5.9 GUI EXAMPLE: ALZHEIMER'S PREDICTION WHERE THE VISIBLE KNOWN LABEL IS MCI 32

5.10 GUI EXAMPLE: STROKE SCAN OUTPUT FROM THE TEMPORARY INTERFACE 32

LIST OF ACRONYMS

AI	Artificial Intelligence
AUC	Area Under the Receiver Operating Characteristic Curve
CNN	Convolutional Neural Network
CPU	Central Processing Unit
CV	Cross-Validation
FP16 / FP32	16-bit / 32-bit floating point
GPU	Graphics Processing Unit
GUI	Graphical User Interface
HD-BET	Deep-learning-based brain extraction tool
MCI	Mild Cognitive Impairment
MRI	Magnetic Resonance Imaging
N4	N4 bias-field correction
NIfTI	Neuroimaging Informatics Technology Initiative file format
OASIS	Open Access Series of Imaging Studies
ONNX	Open Neural Network Exchange
OOF	Out-of-fold
PACS	Picture Archiving and Communication System
ROC	Receiver Operating Characteristic
ROI	Region of Interest
SSL	Self-Supervised Learning
TP / FP / TN / FN	True positive / false positive / true negative / false negative
ViT	Vision Transformer
3D ViT-B	Three-Dimensional Vision Transformer Base

Chapter 1
Introduction

1.1 BACKGROUND AND CONTEXT

MRI is useful for studying brain structure because it captures the whole brain, not a single plane. Each scan is a volume made of voxels, and every voxel has a position and an intensity. Patterns that matter for diagnosis often run across several neighbouring slices, so a model that classifies one slice at a time can throw away useful context. A 3D model reads the scan as one connected volume instead. The price is memory and compute: a 3D network needs far more of both.

Deep-learning models can take over parts of medical image analysis, but a model that works on one dataset does not automatically work on another. The same brain can look different depending on the scanner, the acquisition sequence, the scanning protocol, the patient population and the preprocessing. When any of these change between training and use, the model is shown data unlike what it learned from. There is a quieter risk as well. If one class comes mostly from one data source, the model can learn to recognise the source and not the disease.

Foundation models offer a way around the small-data problem. They learn from a large collection of images first and are then adapted to a narrower task. BrainIAC (Brain Imaging Adaptive Core) is one such model, built for general brain MRI analysis. Its authors pretrained it with self-supervised contrastive learning and tested it on several downstream MRI tasks [1]. We used the BrainIAC 3D Vision Transformer as the starting point for our four-class experiment. Our 60 labelled scans and our fine-tuning are separate from anything in that publication.

We also wanted the workflow to be usable without someone opening a notebook and running cells by hand. The temporary interface lets a user pick or upload a NIfTI volume, run inference, read the score for each class and scroll through axial, coronal and sagittal slices. The plan is to run this interface on a local device: an NVIDIA Jetson module with a touchscreen. So far we have exported the model to ONNX and built a CPU-based interface. The Jetson and touchscreen steps are still ahead of us.

1.2 MOTIVATION

Our first question was simple. Could a pretrained 3D representation be fine-tuned to four labels when we only had 15 scans per label? That is nowhere near enough for clinical use, but it is enough to exercise every engineering stage: loading the model, fine-tuning it, evaluating it, exporting it and wiring it to an interface. Working at this

scale also exposes problems in the data and in the evaluation early, before anyone invests in a larger study.

The second reason was to move past the training notebook. Once the model is exported to ONNX and called from a GUI, we can check how the input is prepared, whether the output comes back in the right format and how long each visible step takes. Deploying on a Jetson raises further questions about memory, runtime compatibility, power draw and heat. A desktop screenshot cannot answer any of those.

Finally, a single accuracy figure was never going to be enough. Class-wise recall and F1, the confusion matrix, ROC and precision-recall curves and the way the data were split each show a different side of the model. Our first held-out test had just nine scans, and the model missed all three Alzheimer's cases in it. A later package reports much better cross-validation numbers, but it also points to a possible shortcut: the data source itself may be helping the model tell the classes apart. For that reason, every result in this report is tied to the evaluation record it came from.

1.3 PROBLEM STATEMENT

This project builds and evaluates a prototype that classifies preprocessed 3D brain MRI volumes as Normal, MCI, Alzheimer's disease or Stroke using a BrainIAC-based 3D Vision Transformer. The classifier is also connected to an interface meant for eventual local use. For that pipeline to be trustworthy, three things have to agree: the model artifact, the preprocessing steps and the mapping from output indices to class names. The evaluation also has to be reported in enough detail that a reader can see what was tested and what is still uncertain.

On the evidence we have, the prototype is not ready for clinical use. The dataset holds only 60 scans, and we cannot rule out that the class labels are tangled up with differences in acquisition or source. The first test used only nine images, and the later cross-validation figures come from a different experiment. The class order in the training log also differs from the order in the Jetson package specification, and nobody has yet shown the full touchscreen application running on the target device. These are open technical and evaluation problems. They have to be part of how the results are read.

1.4 RESEARCH QUESTIONS

1. Can the BrainIAC 3D ViT-B backbone be fine-tuned on the supplied 60-volume dataset and exported for a four-class NIfTI classification workflow?
2. What do the held-out test and cross-validation records say about overall and per-class performance, and where do they disagree?

3. Can a GUI show the predicted class, class scores, volume slices and processing times from an ONNX inference pipeline?
4. What further checks are needed before the exported model can be relied on when running on an NVIDIA Jetson with a touchscreen?

1.5 OBJECTIVES

The objectives below separate work that the project artifacts already demonstrate from work that still has to be proven on hardware. A README, a script or a list of commands shows that a deployment step has been planned; whether the step actually works is a separate question.

- Assemble a balanced dataset of 60 preprocessed NIfTI volumes across the Normal, MCI, Alzheimer's and Stroke classes.
- Fine-tune a BrainIAC-based 3D ViT-B model for four-class prediction and study its training and validation behaviour.
- Evaluate the fine-tuned model with accuracy, precision, recall, macro-F1, the confusion matrix, ROC-AUC and precision-recall analysis.
- Export the model to ONNX and document the input and output tensor shapes, the output order and the preprocessing it expects.
- Build a GUI prototype for loading a scan, showing classification scores and timing, and viewing the volume in three planes.
- Prepare a Jetson inference package and plan the remaining work: building a TensorRT engine, benchmarking it and adding touchscreen support.
- Identify the risks: a small sample, source or domain bias, overfitting, class-index errors and misuse of the tool.

1.6 SCOPE AND BOUNDARIES

The experiment uses four labels, and each label has 15 NIfTI volumes in its own folder. The Alzheimer's, MCI and Normal files follow the OASIS-1 naming pattern (OAS1_####_MR1) [2], while the Stroke files are named stroke_####.nii. A naming difference does not prove where the scans came from. It does raise a fair doubt about whether all four classes share one source, so we treat source effects as a possible confound throughout.

The software path covers file loading, resizing and normalisation, the BrainIAC 3D transformer, four-class prediction, ONNX export and a Gradio interface. The screenshots show inference running on a CPU through ONNX Runtime. The times shown in them are observations from individual sessions, not results of a controlled

benchmark. The Jetson folder contains an inference script and instructions for building a TensorRT engine, but no latency, power, RAM or GPU measurements from a Jetson were taken in this work.

The classifier has not been connected to any clinical workflow or patient-record system. It has not been clinically validated, and it has not gone through a medical-device risk assessment. Explanation maps, a voice interface, PACS integration, report generation and ROS 2 navigation all appear on the roadmap as possible extensions. None of them is implemented.

1.7 EXPECTED CONTRIBUTIONS

- A worked example of fine-tuning a BrainIAC-based 3D ViT for a four-class brain MRI task with NIfTI inputs.
- An evaluation record that shows how each class behaves and keeps the two evaluation runs apart.
- An ONNX inference artifact and a GUI that show one way to present a 3D volume classifier to a user.
- A Jetson deployment plan covering the model tensor contract, the TensorRT build approach and the hardware tests still required.
- An honest account of the current limitations, including possible source shortcuts and the class-order inconsistency.

1.8 ORGANISATION OF THE REPORT

The rest of the report follows the order in which the work was done. Chapter 2 reviews MRI classification, foundation models, BrainIAC, transformer-based image models, preprocessing and edge inference. Chapter 3 covers the dataset, input preparation, the model, the evaluation method, the ONNX interface and the proposed Jetson design. Chapter 4 describes the implementation and the artifacts it produced. Chapter 5 presents the training curves, the held-out results, the package-reported cross-validation results and the GUI examples. Chapter 6 sums up what was achieved, what the results do and do not show, and what still needs to be done.

Chapter 2

Literature Review

2.1 BRAIN MRI AS VOLUMETRIC DATA

An MRI scan is a three-dimensional structure. Each voxel carries an intensity value, and the file also stores orientation, voxel spacing and a coordinate system. The NIfTI format keeps all of this together and is the usual way to store MRI in neuroimaging work [3]. Cutting a volume into an ordinary stack of 2D images discards much of that geometry, and a volumetric classifier may need it.

Tissue also looks different depending on the MRI sequence. T1-weighted, T2-weighted and FLAIR images each emphasise different tissue properties, so they are not interchangeable inputs. Scanner settings and preprocessing add further variation on top of that. A model tends to do well on images like the ones it trained on and poorly on images acquired or processed another way. Pretraining can help the model learn better features, but it does not remove this problem.

Our GUI shows axial, coronal and sagittal slices next to the model output so a user can look at the input and confirm that it loaded correctly. That is all the viewer is for. Marking lesions, measuring anatomy and radiological reading are outside its scope. A scan can look fine on screen and still have registration, orientation or skull-stripping problems, and those need to be checked separately.

2.2 CONVENTIONAL DEEP LEARNING FOR MRI

Convolutional neural networks (CNNs) are common in medical imaging because they pick up local features well: edges, textures and small structures. A 2D CNN works on single slices or on images derived from the volume, so it has to combine slice-level results whenever the task depends on 3D context. A 3D CNN captures that context directly. It is also much heavier on memory and usually needs more labelled data than a small project can supply.

Transfer learning starts from a model trained on some other task and adapts part or all of its weights to the new labels. This is usually more efficient than training from scratch, but only if the pretrained representation suits the new task and the right network structure and checkpoint are actually loaded. Missing or unexpected checkpoint keys are therefore worth investigating. Ignoring them can leave parts of the network untrained without anyone noticing.

2.3 FOUNDATION MODELS AND SELF-SUPERVISED LEARNING

A foundation model learns a general set of features that can be reused across many downstream problems. In medical imaging this matters because labels are slow and expensive to collect. Self-supervised learning fits this setting well. It trains an encoder from unlabelled data, and the encoder's features can then be reused for several tasks.

Contrastive learning is one way to do self-supervised training. In SimCLR, two augmented views are made from each input, and the model learns to place views of the same input closer together in representation space than views of different inputs [4]. With MRI the augmentations need care, since an aggressive transform can erase exactly the information a label depends on. Tak et al. [1] used contrastive self-supervised learning on 3D MRI and then adapted the encoder to specific downstream tasks.

A pretrained encoder does not guarantee good results on new data. Fine-tuning on a small dataset can still overfit, and the model can latch onto accidental links between the label and the way a scan was captured. If each class folder came from a different source or used a different naming convention, fine-tuning could reward the model for spotting the source. For this reason we judge our model on our own local evaluation, not on numbers published for other MRI datasets.

2.4 BRAINIAC: GENERAL-PURPOSE BRAIN MRI FOUNDATION MODEL

Tak et al. [1] describe BrainIAC as a general-purpose foundation model for brain MRI. It learns a representation from a broad collection of scans through self-supervised contrastive learning and is then adapted to several downstream tasks. It suits this project because it was designed for brain MRI volumes rather than natural images. We use it as the starting point for a separate four-class problem with far fewer labelled examples than the original work.

The BrainIAC paper tests the model on MRI-sequence classification, brain-age estimation, mutation prediction, survival prediction, dementia-related prediction, time-to-stroke prediction and tumour segmentation. Those experiments show how widely the authors tested it. They are not results for our classifier. Our data, labels, training procedure and interface are all different, so our model has to be judged on its own evaluation records.

Our model code comes from the BrainIAC repository named in the project roadmap [5]. Reproducing the published setup needs matching architecture definitions, checkpoints and preprocessing. Our training log shows 137 checkpoint tensors loaded, 16 model tensors missing, 12 checkpoint keys unexpected or unmapped, and no shape mismatches. The log cannot tell us whether every missing key is harmless. That has to

be explained from the model-loading code before we can call the implementation fully reproducible.

2.5 VISION TRANSFORMERS AND CONTRASTIVE LEARNING

A Vision Transformer splits an image into patches, turns each patch into a vector and passes the sequence through transformer layers that learn how the patches relate to one another [6]. For 3D MRI the patches become small cubes. According to our model-loading log, the 3D ViT-B has 12 layers, an embedding size of 768 and a patch size of 16. A $96 \times 96 \times 96$ volume is cut into six patches along each axis, which gives $6 \times 6 \times 6 = 216$ patches.

Self-attention lets every token draw information from every other part of the volume, so the model can relate regions that are far apart. The cost grows with the number of tokens, which makes processing slower. We load the pretrained BrainIAC weights as the starting point and then train the model on our four labels. The project files do not record every training setting, so the notebook itself needs to state the optimiser, learning rate and random seed.

The BrainIAC authors chose ViT-B after comparing several pretrained encoders. We use the same model family with a fixed input size of 96 cubed. A fixed shape keeps execution and ONNX export simple, but it also means every input must be prepared the same way. Resizing a raw scan to 96 cubed is not enough on its own to make it a valid input.

2.6 RELATED NEUROIMAGING FOUNDATION MODELS

NeuroSTORM is a foundation model for functional MRI [7]. fMRI adds a time dimension to the three spatial ones, and NeuroSTORM is designed around that spatio-temporal structure. It shares the general idea of a neuroimaging foundation model with our work, but we do not use it. Our pipeline runs on preprocessed structural 3D MRI and uses BrainIAC.

Neuroimaging models represent the brain in several ways: CNN feature maps, transformer tokens, masked-autoencoder features and region-of-interest connectomes. Atlas-based methods shrink the input a model has to handle, though they can lose fine spatial detail. Models that work on the full volume keep more voxel-level structure but need a stable input shape and more compute. Because we work on full volumes, the preprocessing contract matters a great deal for deployment.

2.7 PREPROCESSING AND DOMAIN SHIFT

MRI scans normally go through a preparation pipeline before analysis. The Jetson README lists N4 bias-field correction, resampling to 1 mm voxels, rigid registration to a template head scan and skull stripping with HD-BET. After that, the inference script resizes the prepared volume to 96 × 96 × 96 and normalises the non-zero intensities. These are two separate stages, and they should be documented separately. Open-source toolkits such as MONAI [8] provide many of these transforms for deep-learning pipelines.

If training and inference use different preprocessing, the model receives different inputs from the same scan. Being readable as a NIfTI file is therefore not enough. A scan has to meet the model's registration and skull-stripping requirements too. The GUI carries a warning about this, and users should be told plainly what kind of volume the model expects: a prepared T1-weighted structural scan.

The dataset also raises a question about where the scans came from. The Normal, MCI and Alzheimer's scans follow OASIS-1 naming, and the Stroke scans use a different pattern. The Jetson package includes a shortcut check, and it finds near-perfect one-vs-rest AUC for Normal and Stroke using nothing more than simple image statistics. That suggests the scans may come from different sources. Until a source-balanced dataset is tested, we cannot say these results come only from disease-related features.

2.8 EDGE AI AND MODEL INTERCHANGE

Edge AI means running inference on or near the device that holds the data, instead of sending the data to a remote service. For this project, local inference would cut the dependence on a network and keep scans on the device. The trade-off is that the Jetson's compute, memory, software stack, power budget and thermal limits all constrain the final system. The design has to be tested on the real hardware. Desktop CPU timings do not carry over.

ONNX stores a trained model as a portable computation graph that compatible runtimes can execute [9]. Our artifact, brainiac_multiclass.onnx, takes an input named mri with shape [1, 1, 96, 96, 96] and returns an output named logits with shape [1, 4]. The application applies softmax to the logits and picks the highest score as the predicted class. The class order has to match the model's output indices at every stage of the pipeline.

A model that exports cleanly and runs fast can still give wrong answers. The ONNX output has to be compared against the original framework on known scans, checking both the logits and the predicted labels. If TensorRT FP16 is used, the

comparison has to be repeated, since lower precision can shift the numbers. Latency, memory and power figures for the Jetson should only be reported once they have been measured on the device.

2.9 RESEARCH GAP AND PROJECT POSITIONING

The literature supports starting from a pretrained representation. It says nothing about whether this particular four-class system works. Our contribution is mainly practical. We adapted a 3D encoder, exported an inference model and connected it to a GUI, and we documented where the current data fall short. On the scientific side, a small dataset that may be confounded by source cannot support any claim of clinical generalisation. On the engineering side, TensorRT execution and touchscreen use still have to be shown on the target device.

2.10 COMPARISON OF SELECTED MODEL PARADIGMS

Table 2.1 compares the model families discussed above and notes the main limitation of each for a project of this size.

Table 2.1. Comparison of selected brain-imaging model paradigms

Paradigm	Representation	Potential strength	Main limitation for this project
2D CNN	Individual slices or 2D projections	Lower compute and broad tooling support	May lose through-plane context; needs slice aggregation
3D CNN	Full volumetric patches	Local spatial patterns across 3D anatomy	Memory cost and sensitivity to small labelled datasets
3D ViT / BrainIAC	Volumetric patch tokens with transformer layers	Pretrained transferable representations and long-range interactions	Needs standardised 3D input; downstream evaluation remains data-limited
ROI / connectome model	Atlas-derived regional measurements	Compact features and efficient modelling	May discard voxel-level spatial information
NeuroSTORM-style fMRI model	4D spatio-temporal functional data	Explicit temporal modelling for fMRI	Different imaging modality; not implemented in this project

2.11 SUMMARY

BrainIAC is a sensible starting point for a 3D brain MRI experiment, but our model still has to be evaluated on its own terms. Before we can draw firmer conclusions we need consistent preprocessing, a verified checkpoint and class mapping, and a

validation design that separates subject effects from source effects. Those requirements shaped the system design in the next chapter.

Chapter 3

Methodology and System Design

3.1 SYSTEM OVERVIEW

We split the system into two connected parts. The training workflow takes labelled, preprocessed NIfTI volumes, fine-tunes the model, evaluates its predictions and saves the model artifacts. The application workflow prepares a model-ready volume, calls the exported model and shows the result next to slices of the scan. A third stage, TensorRT inference on an NVIDIA Jetson with a touchscreen, is planned but not yet verified.

Keeping training apart from routine inference means the interface never has to run the notebook. It loads a saved model and follows a fixed preprocessing and output contract. This makes the application easier to package, as long as the model version, the input normalisation and the class-index order stay the same everywhere. If any of them drift between training and the GUI, the interface will still run, but it may show the wrong label. Figure 3.1 shows the architecture and Figure 3.2 the end-to-end workflow.

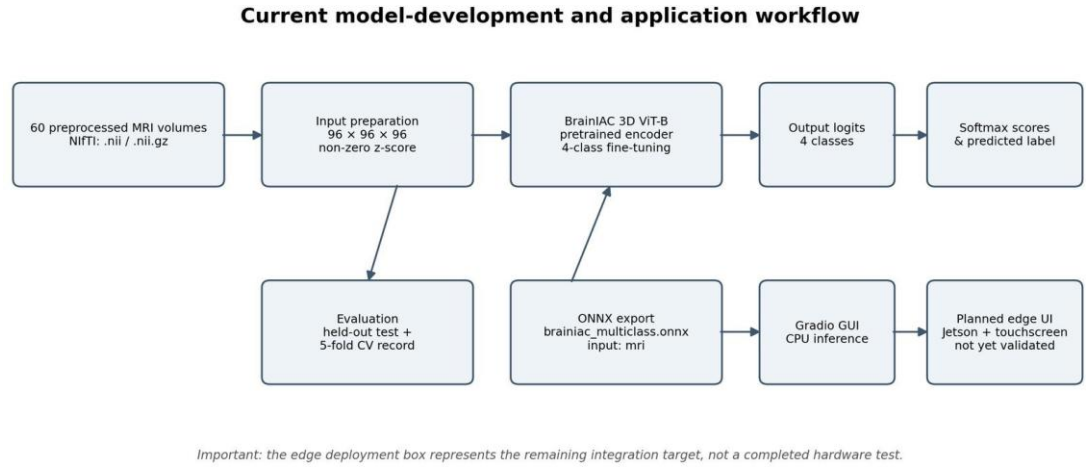


Figure 3.1. Overall system architecture and implementation status

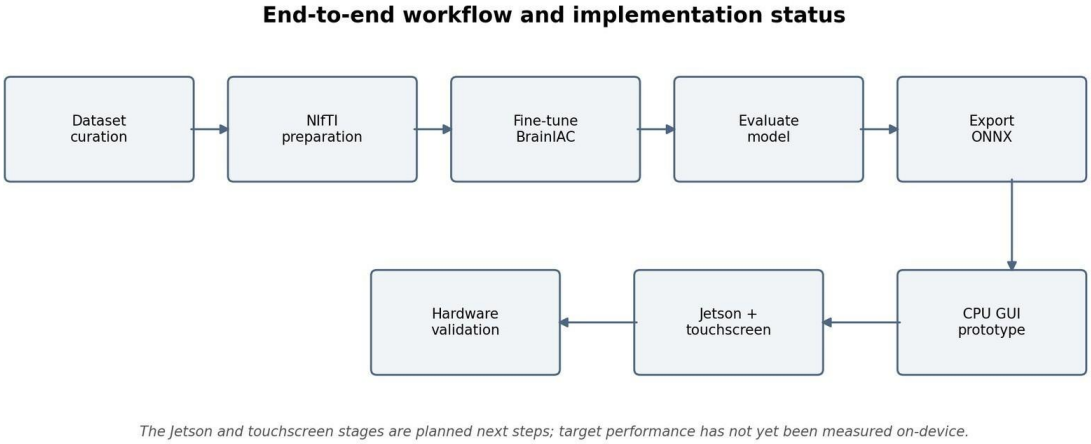


Figure 3.2. End-to-end project workflow

3.2 DATASET CURATION AND LABELS

The by_class.zip archive [10] holds 60 NIfTI volumes in four folders, alzheimers, mci, normal and stroke, with 15 files in each according to the notebook log (Table 3.1). Equal counts make the class distribution easy to describe. They say nothing about whether the groups are comparable in age, sex, severity, scanner, site or subject identity, and the current manifest does not contain enough information to check.

The filename patterns are worth recording, but they are not full provenance. The first three folders use OASIS-1-style names, while the Stroke scans have names like stroke_0004.nii. The package's shortcut check turns this from a naming detail into a real concern. A stronger follow-up study would include every target class from more than one source, or use source-aware evaluation, so the source cannot stand in for the label.

The training notebook takes labels from the folder names and records the mapping alzheimers: 0, mci: 1, normal: 2, stroke: 3. The later Jetson package lists the classes as Normal, MCI, Alzheimer's, Stroke. The export code may have remapped the outputs, or the two sets of files may come from different versions. The files we have do not settle which. We keep the discrepancy in view and treat confirmation against labelled reference scans as a requirement for release.

Table 3.1. Class-wise dataset inventory

Class directory	Volumes	Example filenames
normal	15	OAS1_0069_MR1.nii.gz; OAS1_0085_MR1.nii.gz
mci	15	OAS1_0022_MR1.nii.gz; OAS1_0023_MR1.nii.gz
alzheimers	15	OAS1_0028_MR1.nii.gz; OAS1_0316_MR1.nii.gz
stroke	15	stroke_0001.nii; stroke_0004.nii
Total	60	—

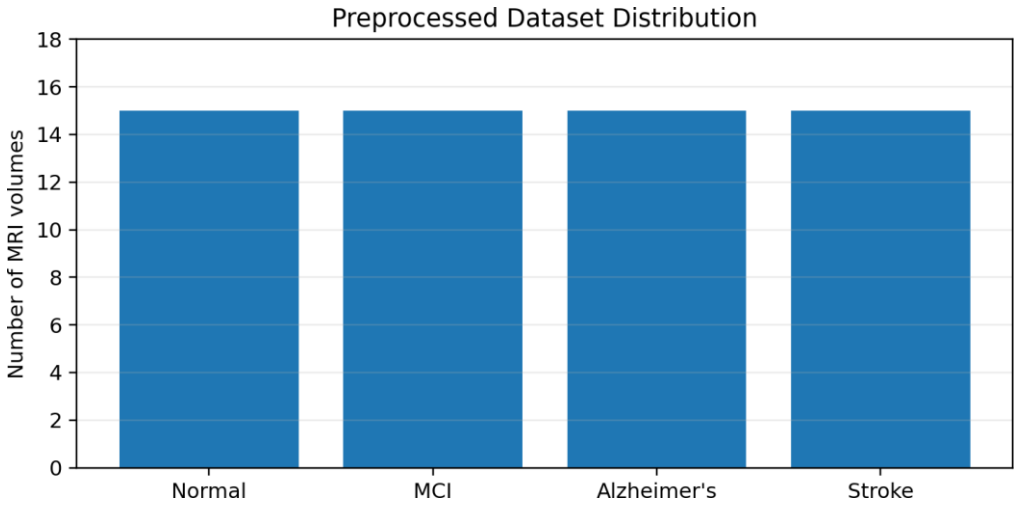


Figure 3.3. Class distribution of the preprocessed scan bundle

3.3 INPUT PREPARATION

The GUI accepts .nii and .nii.gz files and warns that they should already be skull-stripped and registered to the BrainIAC atlas. The Jetson README describes how to get a raw T1 volume to that state: N4 bias correction, resampling to 1 mm, rigid registration to temp_head.nii.gz and HD-BET skull stripping. All of that has to happen before the short inference function is called.

Inside jetson_infer.py, runtime preparation converts the volume to 32-bit floating point, replaces any non-finite values with zero and resizes it to 96 × 96 × 96 with trilinear interpolation (align_corners=False). It then computes the mean and population standard deviation over the non-zero voxels and z-scores those voxels, leaving the zero background as it is. If the standard deviation is zero, the script divides by 1.0 to avoid a division by zero. This describes the inference script. The training notebook still needs to be checked to confirm it does the same thing.

The model expects an input tensor of shape [1, 1, 96, 96, 96]: one volume, one channel and three spatial axes. A volume with very few non-zero voxels or odd intensity values may not normalise in any meaningful way. The current function copes with non-finite values and a zero standard deviation, but the application around it should also reject empty volumes and check orientation, shape and preprocessing status before running inference.

3.4 MODEL ARCHITECTURE

The classifier uses a BrainIAC-based 3D Vision Transformer Base as its backbone, with a task-specific head for the four classes. The model-loading log records a depth of 12, an embedding size of 768, a patch size of 16 and 216 patches for the 96-cubed input. These values come from the log, not from a fresh rebuild of the full architecture.

In outline, the preprocessed volume is cut into 3D patches, the patches are embedded, and the sequence passes through the transformer encoder blocks. A classification head turns the result into four logits. Softmax converts the logits into scores that add up to one, and the class with the highest score is shown as the prediction. These percentages are model scores. Reading them as calibrated probabilities, or as anyone's risk of disease, would be a mistake.

The checkpoint-loading log reports 137 tensors loaded, 16 model tensors missing, 12 checkpoint keys unexpected or unmapped and no shape mismatches. The missing keys include the attention QKV biases and the classifier parameters, and some of the unexpected keys belong to projection-head layers. This pattern could simply reflect the difference between the pretraining model and the downstream model, but the log alone

cannot confirm that every mismatch is expected. The loading report should be checked against the architecture code and kept with the final experiment record.

3.5 FINE-TUNING PROCEDURE

The Kaggle workflow [11] reads the class folders, assigns labels, makes the data splits, initialises the BrainIAC-based model, swaps its output layer for a four-class head and trains for ten epochs. As it trains, it logs loss and classification metrics, and at the end it saves checkpoints and evaluation outputs. According to the Jetson package metadata, a final full-mode ten-epoch fit was run on all 60 volumes after cross-validation. That full-data model is the one meant for inference. The cross-validation figures should come from predictions on the scans held out in each fold.

To make the experiment fully reproducible, the notebook should record the optimiser and its settings, the learning rate, weight decay, batch size, random seed, augmentations, the rule for choosing a checkpoint and which parameters were trainable. Not all of this can be recovered from the screenshots and the compact package. The roadmap mentions freezing the feature extractor, but the package metadata says mode=full. Only the original training code can tell us whether freezing was ever used or was just part of the early plan.

3.6 DATA SPLITS AND VALIDATION

The early experiment used 42 scans for training, 9 for validation and 9 for testing. The training set had 11 MCI, 11 Stroke, 10 Normal and 10 Alzheimer's scans. The validation set had three Normal scans and two from each of the other classes. The test set had three Alzheimer's scans and two from each of the rest (Figure 3.4). With only nine test cases, a single changed prediction moves accuracy by 11.11 percentage points. These results are a small diagnostic check, not a stable estimate.

The later Jetson package reports five-fold subject-grouped cross-validation, with summary metrics computed from the pooled out-of-fold predictions. Grouping by subject keeps repeat scans of one person from landing in both the training and held-out data. The package gives the resulting metrics, but the compact archive does not include all the fold manifests needed to recompute them. We therefore label these as package-reported results. We did not rerun the folds ourselves.

The two evaluation records should stay separate until the gap between them can be explained. The nine-scan holdout belongs to the first notebook run. The larger figures belong to the later package's cross-validation metadata. Different splits, checkpoints, training code or class mappings could account for the difference, and the artifacts do not show which. Before quoting a single headline number, the team should

archive the split manifests, per-scan predictions, code version and metric script for each run.

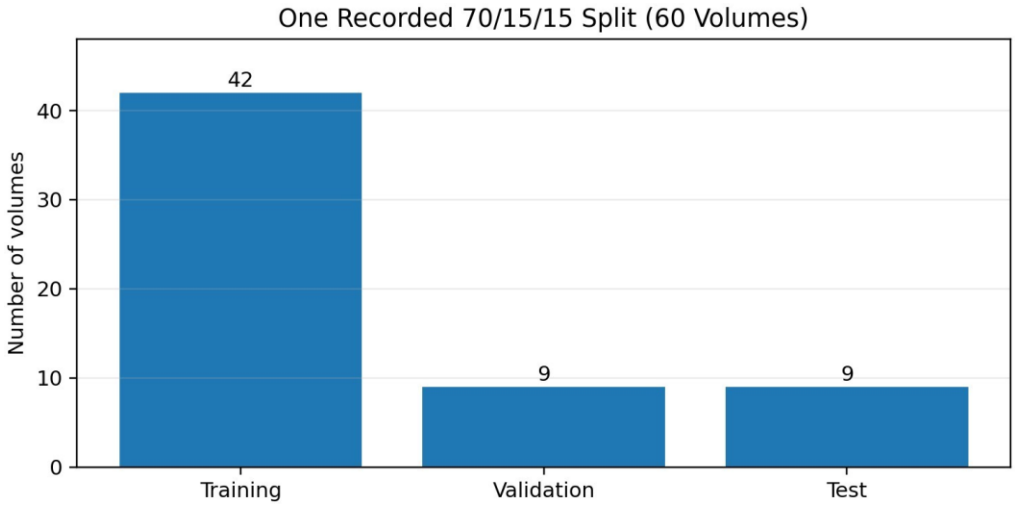


Figure 3.4. One recorded training/validation/test split from the initial experiment

3.7 EVALUATION METRICS

Accuracy is the fraction of samples classified correctly. Precision asks, for a given class, how many of the samples assigned to it really belong to it. Recall asks how many of the class's true samples the model found. F1 combines the two into a single number. Macro averages give every class the same weight, whereas weighted averages scale each class by its number of examples. Balanced accuracy is the mean of the class recalls. It helps when classes are unequal in size, though it is still unstable when there are very few samples.

A ROC curve plots the true-positive rate against the false-positive rate as the decision threshold moves. ROC-AUC measures how well the scores rank positive cases above negative ones, here in a one-vs-rest setting. A precision-recall curve shows the trade-off between precision and recall, and average precision summarises it. Neither replaces the confusion matrix, because the interface picks its label with an argmax. When a class has only a few examples, these curves can jump sharply if one sample moves. Table 3.2 lists the metrics and the caution each needs here.

Table 3.2. Evaluation metrics and limitations

Metric	Definition / use	Caution in current setting
Accuracy	Correct predictions divided by number of test samples	Highly unstable with nine cases
Precision	TP / (TP + FP)	Undefined when there are no predicted positives; implementations may report zero
Recall / sensitivity	TP / (TP + FN)	Missed Alzheimer's cases are especially visible in class-wise recall

Metric	Definition / use	Caution in current setting
F1-score	2 × precision × recall / (precision + recall)	Depends on both false positives and false negatives
Macro-F1	Unweighted average of class F1 values	Treats each class equally; appropriate for balanced class reporting
Balanced accuracy	Mean recall across classes	May still be unstable when each class has few examples
ROC-AUC	Area under one-vs-rest ROC curve	Can be high despite poor thresholded predictions
Average precision	Area-style summary of precision-recall ranking	Sensitive to very small class support and prevalence

3.8 ONNX EXPORT AND INFERENCE CONTRACT

The Jetson archive [12] contains brainiac_multiclass.onnx, spec.json, expected.json, sample_input.bin, sample_scan.nii.gz and jetson_infer.py. The specification names the input mri with shape [1, 1, 96, 96, 96] and the output logits with shape [1, 4] (Table 3.3). The script applies softmax and takes the highest-scoring output. The package lists the classes as Normal, MCI, Alzheimer's, Stroke. The reference JSON is a useful check for one Normal scan, but one scan cannot confirm every output label.

There is no prebuilt TensorRT engine in the archive. The README tells the team to build the engine on the Jetson, compare its output with the reference and look at runtime resource use. Building on the device keeps the engine matched to that hardware and software stack. Until those steps are run and logged, TensorRT execution and its speed are planned work. FP16 output should be checked against FP32. A successful build does not mean the two give the same results.

Table 3.3. Model input/output contract recorded in the Jetson package

Property	Recorded value
Model file	brainiac_multiclass.onnx
Input name	mri
Input shape	[1, 1, 96, 96, 96] float32
Output name	logits
Output shape	[1, 4] logits
Class order in package spec	normal, mci, alzheimers, stroke
Post-processing	softmax(logits); predicted class = argmax
Input preparation in inference script	Trilinear resize to 963; z-score of non-zero voxels
Reference test	expected.json; one Normal scan example

3.9 GUI DESIGN

The temporary interface is written in Python with Gradio. On the left, the user picks a demonstration scan or uploads a preprocessed .nii or .nii.gz file and presses Analyse. The results panel shows the predicted label, a bar for each class score, the filename and, where the sample has one, its known label. Separate axial, coronal and sagittal panels display the volume, each with a slider for moving through the slices.

The interface also reports preprocessing and inference times and carries a warning that the prototype is not a medical device. In the screenshots, ONNX Runtime runs on a CPU, with inference taking roughly 0.3 to 1.0 seconds and preprocessing roughly 0.08 to 0.41 seconds. Those numbers describe the captured sessions only. The machine, the warm-up procedure and repeat-run statistics were never recorded, so they cannot be treated as a benchmark.

The input warning should stay visible in every later version. Users need to know that the model expects a prepared volume, not any MRI file, and that its output is a research prediction, not a diagnosis. The known-label badge is helpful for the selected demonstration scans, but it should not appear for new uploads unless their true label really is known. The model version and preprocessing status should also be easy to find.

3.10 TARGET JETSON ARCHITECTURE

The intended end state is a local application running on a Jetson and operated from a touchscreen. The roadmap names the Jetson AGX Orin and proposes a 1 TB NVMe SSD, a touchscreen, and optionally a USB camera and microphone. The camera, microphone and mobile-robot extensions are not needed for the core classifier demonstration. What matters now is launching the GUI on the actual Jetson and checking the full scan-to-result workflow from the touchscreen.

The package includes an ONNX model and a TensorRT inference script. The README describes building the engine on the device, testing it with trtexec, timing repeated runs and watching the system with tegrastats or jtop. We found no on-device logs showing that any of these tests were done. This report therefore describes a prepared deployment route, not a finished Jetson implementation.

3.11 END-TO-END ALGORITHM

Algorithm 3.1 sets out the inference path in the supplied package. It assumes registration and skull stripping have already been done, because the short inference function does not perform them.

```
Algorithm 3.1: Inference path for one scan
INPUT: model-ready NIfTI volume V, ONNX/TensorRT model M,
        ordered class names C
1. Load V from .nii or .nii.gz.
2. Convert the voxel array to float32; replace NaN/Inf with zero.
3. Resize V to 96 x 96 x 96 using trilinear interpolation.
4. Find non-zero voxels; compute mean mu and population std sigma.
5. Normalise non-zero voxels as (V - mu) / max(sigma, eps);
   keep zero voxels at zero.
6. Add batch and channel axes to get [1, 1, 96, 96, 96].
7. Run M to obtain four logits z.
8. Compute p = softmax(z); k = argmax(z).
9. Display C[k], scores p, scan views and timing.
OUTPUT: predicted class and four model scores (not a diagnosis)
```

3.12 DESIGN RATIONALE AND RISKS

A pretrained 3D encoder gives the project a much better starting point than learning every image feature from 60 labelled scans. Keeping preprocessing separate from inference makes each stage easier to inspect, though it also means training and deployment must use exactly the same steps. ONNX lets the model run outside the training notebook, and the GUI makes the workflow easy to demonstrate. Speed is not the hard part. The harder problems are the correct label order, possible source shortcuts, overfitting, input validation and what the displayed scores actually mean.

Chapter 4

Implementation and Experimental Setup

4.1 DATASET AND FILE ORGANISATION

The fine-tuning input is the labelled NIfTI collection in by_class.zip [10]. Each class folder holds 15 files, so the four labels start with equal counts. Appendix 1 lists every filename in the archive. The list is handy for tracking inputs, but it carries no patient-level details and no full acquisition metadata.

The mixed naming patterns call for a provenance check. The Alzheimer's, MCI and Normal folders use OASIS-1-style names [2], and the Stroke folder uses plain numbered names. Equal file counts do not mean the scans were acquired under comparable conditions. The Jetson package's shortcut test also points to a source effect, so the next version of the dataset should draw each class from more than one source, or use a design that can separate source from diagnosis.

4.2 FINE-TUNING WORKFLOW

The team's preprocessing and fine-tuning work is recorded in a Kaggle notebook [11]. The screenshots taken for this report show the class counts, the label mapping, the data split, the checkpoint-loading messages, the ten training epochs and the evaluation plots. We treat those screenshots and the saved output files as the record of the first run. Anyone reproducing the experiment would still need the full executable configuration, which screenshots cannot provide.

Over the ten logged epochs, training loss falls from 1.4232 to 0.6204 and training accuracy climbs from 0.381 to 0.738. Validation accuracy sits at 0.444 for epochs 1 to 9 and then drops to 0.333 at epoch 10. Validation loss is lowest, at about 0.8643, in epoch 5, and then rises to 1.1224. The model keeps getting better on the training data while the validation metrics stay flat or worsen. With only nine validation scans this is a warning sign, not a precise measure of generalisation.

4.3 EVALUATION ARTIFACTS

The training archive contains the history, test metrics, classification report, confusion matrix, per-scan predictions, ROC and precision-recall plots, per-class metrics, accuracy and loss charts, and the input and split manifests. The compact jetson_package_4class.zip has a different job. It holds the ONNX model, its specification, an example input and output, a sample NIfTI file, the inference code and deployment notes. The two bundles support different parts of the workflow and are not

interchangeable copies of the same results. Table 4.1 lists each artifact and what the evidence shows about it.

Table 4.1. Artifact and software inventory

Artifact / source	Role	Status shown by supplied evidence
Kaggle fine-tuning notebook	Preprocessing and training workflow	Notebook URL and screenshots provided
by_class.zip	Four-class preprocessed dataset	60 files; 15 per class verified from archive
Initial training outputs	Curves, report, confusion matrix, test metrics	Screenshots provided
brainiac_multiclass.onnx	Portable inference graph	Included in Jetson package
spec.json	I/O shape, class order, metrics metadata	Included in Jetson package
expected.json and sample_input.bin	Reference output / smoke test	Included in Jetson package
jetson_infer.py	TensorRT inference script	Included in Jetson package
TensorRT engine	Device-specific optimised model	Not included; README says to build on device
Touchscreen GUI running locally	Final interface integration	Not evidenced as completed

4.4 ONNX PACKAGE AND INFERENCE

The Jetson package [12] holds an ONNX model of about 353 MB, along with the model specification, the expected-output JSON, a binary input sample, an example NIfTI scan, the inference script and a README. According to the file listing, the compressed archive is about 318 MB. ONNX Runtime can load the graph, and TensorRT is the intended route for optimised execution on the device. The included scan shows the expected input format. It tells us nothing about whether that sample represents a clinical population.

The inference script allocates a CUDA output tensor with one score per class, binds the input and output buffers to TensorRT and runs execution on the current CUDA stream. It runs a warm-up step and then reports mean, median and p95 engine latency over repeated calls. Those numbers cover engine execution only. To know how long a user actually waits, NIfTI loading and preprocessing must be timed separately and added on.

The README recommends building the TensorRT engine on the Jetson and testing it with trtexec. It also suggests comparing the result against the expected logits and class, falling back to full precision if the FP16 differences are too large, and

checking memory use, GPU load and power. Those are instructions for testing; nothing has been measured yet. The final record should include the exact Jetson model, the JetPack and TensorRT versions, the power mode, the warm-up count, the number of timed runs and the measured values.

4.5 GUI IMPLEMENTATION

The interface has two columns. The left column holds the demonstration-scan selector, the file-upload control and the Analyse button. Below them, a warning explains that scans must be skull-stripped and registered, and that known labels are revealed for demonstration records only after inference. The right column shows the predicted class, the score bars, timing information, an optional true-label badge, and axial, coronal and sagittal views.

With this interface a reviewer can follow the pipeline without opening the training notebook. The sliders make it easy to look through different parts of the volume, and the score bars show how the model spreads its output across the four classes, not just which one wins. The bars can still make the model look more confident than it deserves to, because softmax scores are not calibrated probabilities. The screen should call them model scores and keep the research-use warning in view.

Some of the example scans are files the team already knew, and some come from the training set. A correct prediction on one of these shows that the file passes through the software, and nothing more about how the model would do on new patients. Where a cross-validated prediction from a model that never trained on the scan is available, the GUI says so. These screenshots document how the interface behaves. They are not extra independent test cases.

4.6 DEPLOYMENT PLAN AND CURRENT STATUS

Table 4.2 sets out each work package, the evidence available for it and the criterion that would count it as finished.

Table 4.2. Current implementation status and completion criteria

Work package	Evidence / current status	Next completion criterion
Dataset preparation	60 preprocessed NIfTI volumes organised by class	Archive data provenance and subject/source metadata
Fine-tuning	Ten-epoch run and model/evaluation outputs shown	Pin notebook revision and all hyperparameters
Initial evaluation	Nine-scan test metrics and plots available	Keep this run labelled separately from cross-validation
Cross-validation record	Package metadata reports 5-fold subject-grouped OOF metrics	Archive fold assignment and per-scan OOF predictions

Work package	Evidence / current status	Next completion criterion
ONNX export	ONNX model and reference artifacts included	Verify class order and parity on multiple samples
GUI	Gradio prototype with upload, scores, timing and slices	Run locally on Jetson and test touchscreen scaling
TensorRT optimisation	Build instructions and TensorRT script included	Build and benchmark engine on target Jetson
Touchscreen integration	Planned; not evidenced as completed	Demonstrate full local workflow without cloud dependency
Clinical validation	Not performed; outside current prototype claim	Would require external data, ethics approval, calibration and a clinical study

4.7 REPRODUCIBILITY AND QUALITY CHECKS

The following checks are needed before any result from this project is treated as reproducible.

- Archive the exact notebook revision, the model checkpoint hash, the exported ONNX hash, dependency versions and the configuration behind every reported metric.
- Confirm the order of the four labels in the training classifier, the ONNX logits and spec.json. A single example is not enough to establish the mapping.
- Run PyTorch-versus-ONNX parity tests on several inputs covering all four classes, comparing logits, softmax scores and predicted labels.
- Keep the subject-grouped fold assignments and out-of-fold predictions so the reported metrics can be recomputed independently.
- Run a dataset-source shortcut baseline and a source-aware analysis. If class and source turn out to be strongly linked, collect more data before reading high scores as disease recognition.
- Check NIfTI affine and orientation, finite values, non-zero voxels, dimensions and preprocessing history before allowing inference.
- On the Jetson, record the device configuration, warm-up and timed runs, mean and p95 latency, peak RAM, GPU utilisation, power mode and numerical parity results.

4.8 ETHICAL AND SAFETY CONSIDERATIONS

This software is a research prototype and not a medical device. It must not be used to diagnose, rule out or manage Alzheimer's disease, MCI or stroke. The four labels are categories used in this experiment, and a prediction does not establish whether a person

has any condition. The score on screen is not a clinically validated probability. Calibration and independent clinical evaluation would both be needed before it could be read that way.

The dataset consists of medical images, so the team has to follow the usage terms attached to each source. Where the scans come from human participants, the relevant data-use conditions, institutional rules and licences apply. A filename is not proof that a scan has been de-identified. The data should be stored carefully, and raw clinical images should not be sent to outside services without the necessary permission.

The BrainIAC paper and repository are released for research, and the Jetson README describes the model licence as non-commercial academic research. The licences of the fine-tuning data apply as well. Before the system is shared beyond the academic setting, or considered for any commercial use, the team should confirm the terms and keep proper attribution. This report does not claim that permission has been obtained for every possible deployment.

4.9 SUMMARY

The project now covers dataset organisation, model fine-tuning, evaluation outputs from two runs, an ONNX artifact and a CPU-based GUI. The next stage involves more than copying the model to a Jetson. We have to confirm the output-label mapping, compare ONNX results with the training framework, build a device-specific TensorRT engine, time the full inference path and test the GUI on the touchscreen. Chapter 5 looks at the evidence we currently have.

Chapter 5

Results and Analysis

5.1 DATASET AND SPLIT SUMMARY

The prepared dataset has 60 scans, 15 per class. The first notebook split used 42 for training, 9 for validation and 9 for testing, as Table 5.1 shows. The test set held three Alzheimer's scans and two from each other class. With nine cases, one changed prediction moves accuracy by 11.11 percentage points. The result is good for spotting errors in this run, but far too small to predict performance in a wider population.

Table 5.1. Class counts and recorded split

Class	Total scans	Training	Validation	Initial test
Alzheimer's	15	10	2	3
MCI	15	11	2	2
Normal	15	10	3	2
Stroke	15	11	2	2
Total	60	42	9	9

Fifteen files per folder does not rule out a source imbalance. The shortcut check in the Jetson package reports AUC values close to 1.00 for Normal and Stroke using only simple image statistics, and the package puts this down to differences in dataset or acquisition sequence. That is an important caveat for the four-class result. Part of the separation may come from how the scans were collected, not from anything to do with disease.

5.2 TRAINING AND VALIDATION BEHAVIOUR

Over the ten recorded epochs, training loss falls from 1.4232 to 0.6204 and training accuracy rises from 0.381 to 0.738 (Figures 5.1 and 5.2). The validation curves look quite different. Accuracy holds at 0.444 through epoch 9 and then drops to 0.333. Validation loss bottoms out at 0.8643 around epoch 5 and climbs to 1.1224 by epoch 10. The model keeps improving on the training data, and none of that improvement shows up in validation accuracy.

This is what overfitting looks like, although a validation set this small cannot prove it statistically. With nine examples the metric moves in big steps, so the flat accuracy curve should not be read too closely either. A better evaluation would use grouped, stratified folds, choose the checkpoint on the validation fold alone and report how much the scores vary from fold to fold.



Figure 5.1. Training and validation loss over ten epochs (initial run)

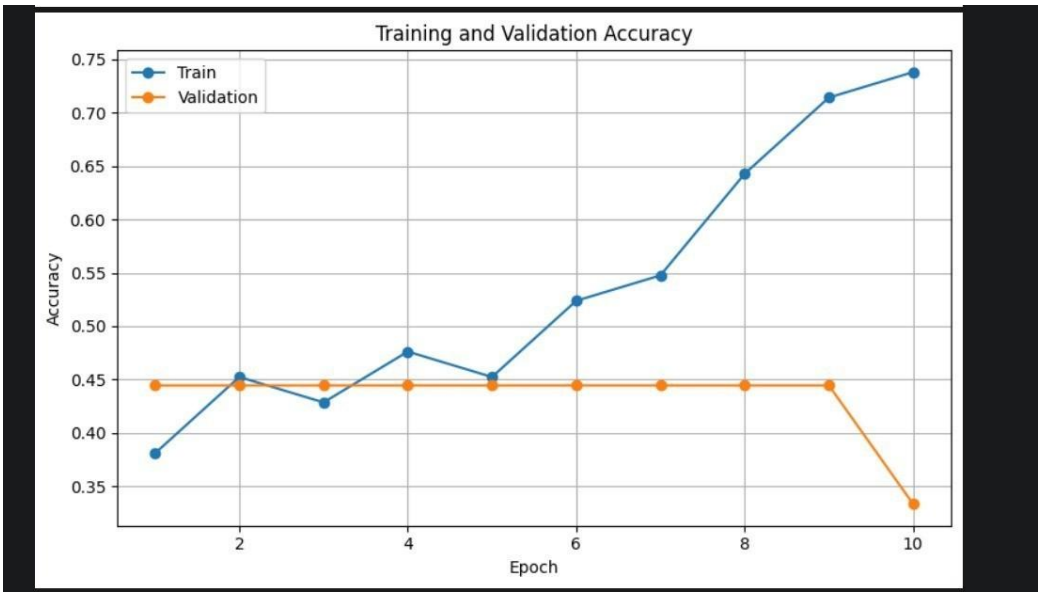


Figure 5.2. Training and validation accuracy over ten epochs (initial run)

5.3 INITIAL HELD-OUT TEST RESULTS

On the first nine-scan holdout, the model reached an accuracy of 0.4444, macro precision of 0.4250, macro recall of 0.5000, macro-F1 of 0.4464, weighted-F1 of 0.3968 and macro one-vs-rest ROC-AUC of 0.7579 (Table 5.2). Four of the nine predictions were correct. A macro recall of 0.50 is just the average of the class recalls, and it hides large differences between classes, as the next section shows.

Table 5.2. Initial held-out test metrics

Metric	Initial held-out value
Accuracy	0.4444 (44.44%)

Metric	Initial held-out value
Macro precision	0.4250
Macro recall / balanced accuracy	0.5000
Macro-F1	0.4464
Weighted-F1	0.3968
Macro one-vs-rest ROC-AUC	0.7579

5.4 CONFUSION MATRIX AND PER-CLASS BEHAVIOUR

The confusion matrix shows where the errors fell (Table 5.3 and Figure 5.3). All three Alzheimer's scans were labelled MCI. Of the two MCI scans, one was right and one was called Normal. The two Normal scans split the same way, one correct and one called MCI. Both Stroke scans were classified correctly. In this split the model mixed up Alzheimer's and MCI, but there are far too few examples to estimate per-class sensitivity with any confidence. Table 5.4 gives the per-class report.

Table 5.3. Confusion matrix counts by actual and predicted class

Actual class	Predicted Alzheimer's	Predicted MCI	Predicted Normal	Predicted Stroke
Alzheimer's	0	3	0	0
MCI	0	1	1	0
Normal	0	1	1	0
Stroke	0	0	0	2

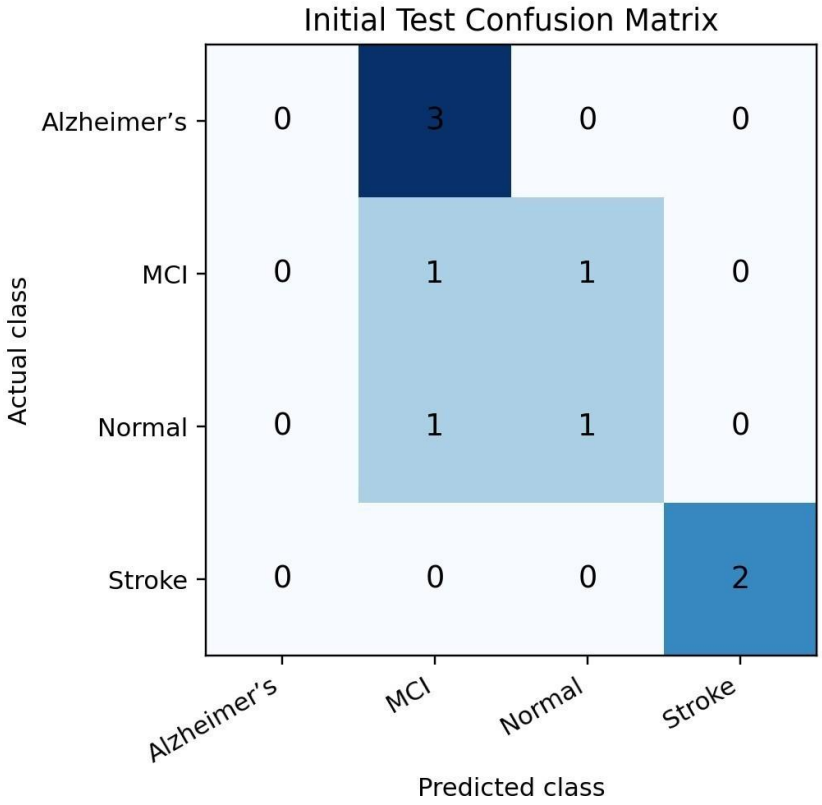


Figure 5.3. Initial held-out test confusion matrix; row totals are 3, 2, 2 and 2

Table 5.4. Initial test classification report by class

Class	Precision	Recall	F1-score	Support
Alzheimer's	0.000	0.000	0.000	3
MCI	0.200	0.500	0.286	2
Normal	0.500	0.500	0.500	2
Stroke	1.000	1.000	1.000	2

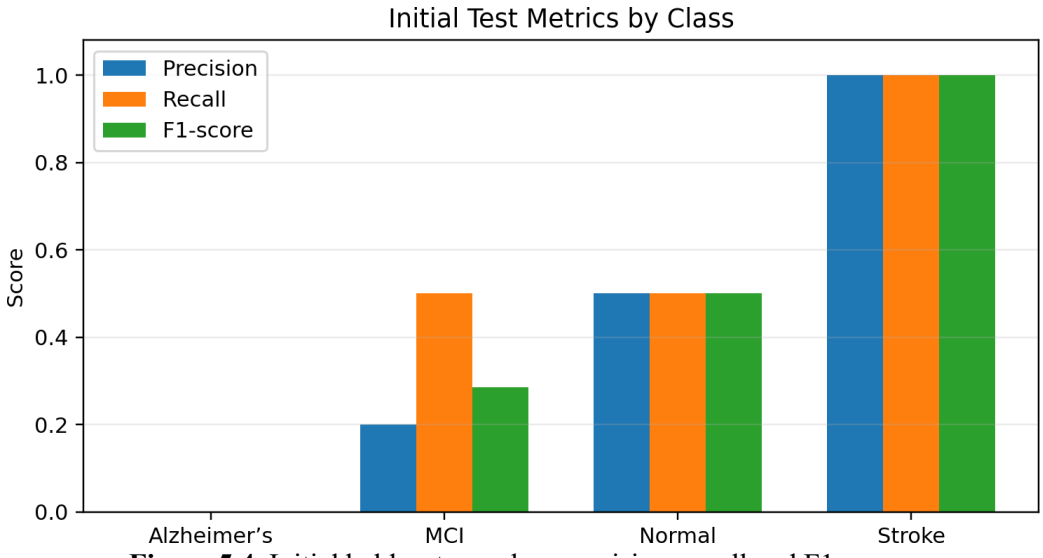


Figure 5.4. Initial held-out per-class precision, recall and F1-score

5.5 ROC AND PRECISION-RECALL ANALYSIS

In the same nine-scan run, the one-vs-rest ROC-AUC was 0.889 for Alzheimer's, 0.429 for MCI, 0.714 for Normal and 1.000 for Stroke (Figure 5.5). Average precision from the precision-recall curves was 0.806, 0.267, 0.450 and 1.000 in the same order (Figure 5.6 and Table 5.5). These figures belong to the first holdout, not to the later cross-validation. ROC-AUC and average precision measure how well the scores rank cases across all thresholds, while the interface picks its class with an argmax. A ranking metric can look respectable even when the final decisions are poor, especially with this few cases.

The perfect Stroke AUC and average precision rest on just two Stroke test scans, so they are highly uncertain. The Alzheimer's AUC of 0.889 does not change the fact that the model's recall for that class was zero in this split. MCI's AUC below 0.5 means its scores ranked the examples badly in this small sample. These plots are useful for deciding what to look into next. They say nothing about clinical performance.

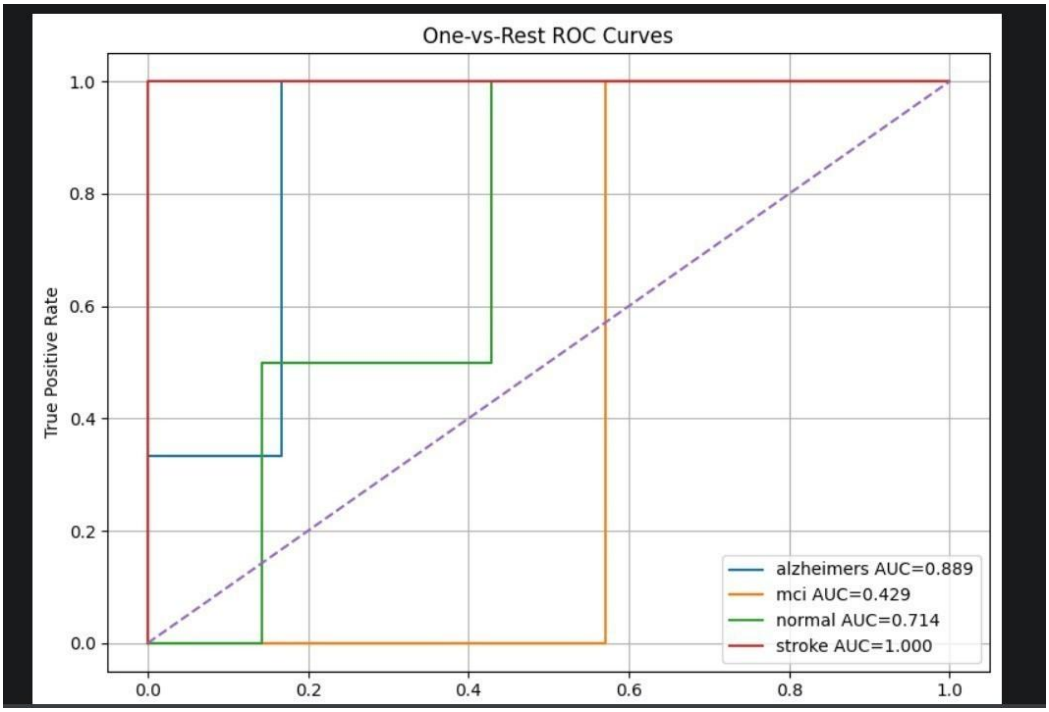


Figure 5.5. Initial held-out one-vs-rest ROC curves and class-wise AUC values

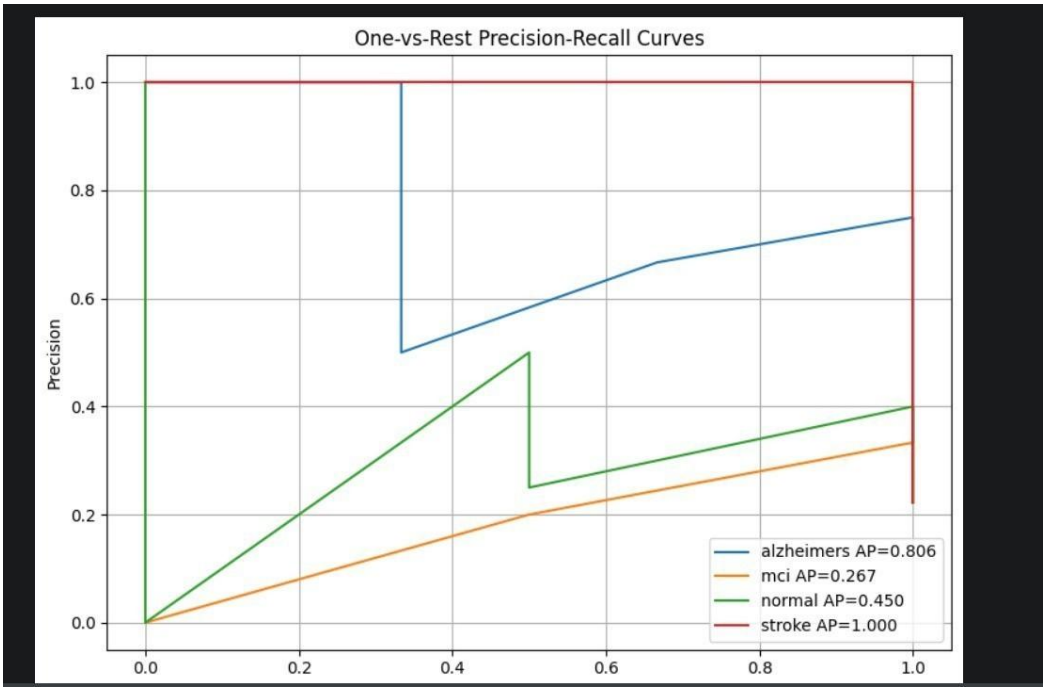


Figure 5.6. Initial held-out one-vs-rest precision-recall curves and average precision values

Table 5.5. Per-class one-vs-rest AUC and average precision

Class	One-vs-rest AUC	Average precision
Alzheimer's	0.889	0.806
MCI	0.429	0.267
Normal	0.714	0.450
Stroke	1.000	1.000

5.6 CROSS-VALIDATION RESULTS IN THE JETSON PACKAGE

The later Jetson package README and spec.json report five-fold subject-grouped cross-validation. The pooled out-of-fold metrics are 0.7833 accuracy, 0.7833 balanced accuracy, 0.7784 macro-F1 and 0.9248 macro-AUC, with a reported 95% interval for macro-AUC of 0.8753 to 0.9678 (Table 5.6). The package also gives per-class values, listed in Table 5.7. This is a different evaluation record from the nine-scan split, so the two sets of numbers are reported separately.

Table 5.6. Cross-validation metrics reported by Jetson package metadata

Metric	Package-reported CV value
Pooled out-of-fold accuracy	0.7833
Balanced accuracy	0.7833
Macro-F1	0.7784
Macro-AUC	0.9248
95% CI for macro-AUC	0.8753–0.9678

Metric	Package-reported CV value
Number of scans	60; 15 per class
Cross-validation	5-fold subject-grouped, pooled out-of-fold

Table 5.7. Per-class metrics from package-reported cross-validation

Class	Precision	Recall	F1-score	One-vs-rest AUC	Support
Normal	1.000	0.933	0.966	0.997	15
MCI	0.600	0.400	0.480	0.836	15
Alzheimer's	0.591	0.867	0.703	0.870	15
Stroke	1.000	0.933	0.966	0.997	15

The package also reports an evaluation limited to Normal, MCI and Alzheimer's, leaving Stroke out. Across those 45 scans it gives 0.7333 accuracy, 0.7333 balanced accuracy, a macro-F1 of 0.7227 and a macro-AUC of 0.86. The README describes this as a fairer within-source comparison. The lower macro-AUC, together with the shortcut baseline's near-perfect Normal and Stroke scores, supports the worry that source or sequence differences are inflating the four-class result. The three-class figure is still preliminary, and it does not remove every possible source difference among the remaining groups.

5.7 RESULT RECONCILIATION AND LABEL-ORDER RISK

We have two sets of results that must not be blended together. The first holdout reached 44.44% accuracy and a macro-F1 of 0.4464 on nine scans. The later package reports 78.33% out-of-fold accuracy and a macro-F1 of 0.7784 from five-fold cross-validation. The artifacts do not tell us why the two differ. The splits, the training run, the checkpoint, the class mapping or the evaluation code may all have changed. To settle it, the team needs the per-scan predictions and split manifests for both runs, along with the code version and metric calculation used for each.

The class-index order is a separate problem. The first notebook lists alzheimers: 0, mci: 1, normal: 2, stroke: 3, while the Jetson package lists Normal, MCI, Alzheimer's, Stroke. If the indices are not remapped consistently, the model can output perfectly valid logits and the GUI can still print the wrong class name. One reference Normal scan works as a smoke test, but it cannot confirm all four mappings. Every output has to be tested with labelled examples from all four classes and compared with the original PyTorch model.

According to the package metadata, the inference model was refit on all 60 volumes after cross-validation. That is a normal packaging step, but it means every one of the 60 scans has been used to train the final artifact. Its predictions on those scans cannot be reported as independent test results. The out-of-fold predictions are the right internal estimate from the cross-validation experiment, and a separate external dataset is still needed for a real test.

5.8 GUI DEMONSTRATION

The GUI screenshots confirm that the application can load a NIfTI file, prepare it, call the exported model and show scores alongside orthogonal slice views (Figures 5.7 to 5.10). Reliability is another matter. The Normal example, normal_0005.nii, shows 96%. The MCI example, OAS1_0022_MR1.nii, shows 84% and matches its known label. The OAS1_0023_MR1.nii example is predicted as Alzheimer's even though its known label is MCI. The Stroke example, stroke_0004.nii, shows 95%. We kept the wrong prediction in on purpose, because it shows that a high score on screen does not guarantee a correct label.

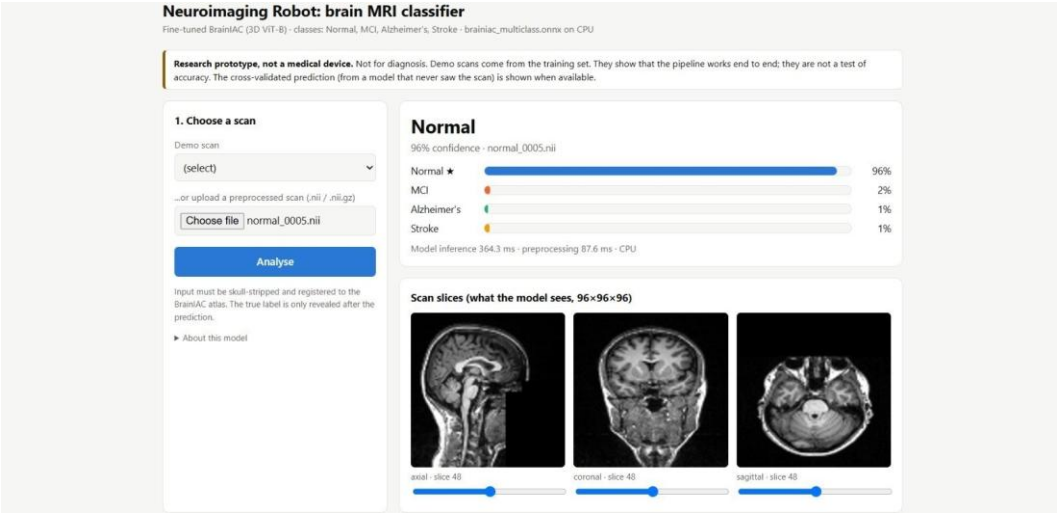


Figure 5.7. GUI example: Normal scan output from the temporary interface

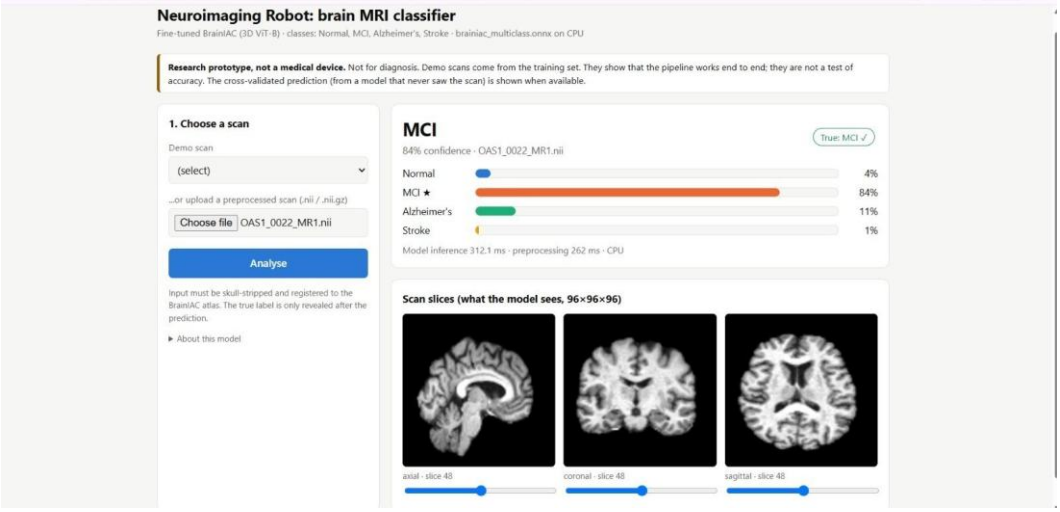


Figure 5.8. GUI example: MCI prediction for a scan with known MCI label

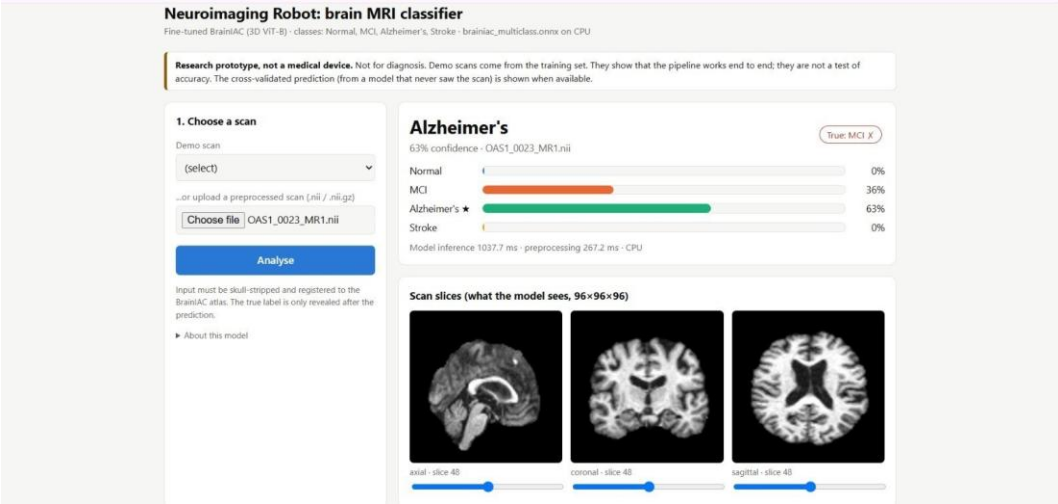


Figure 5.9. GUI example: Alzheimer's prediction where the visible known label is MCI

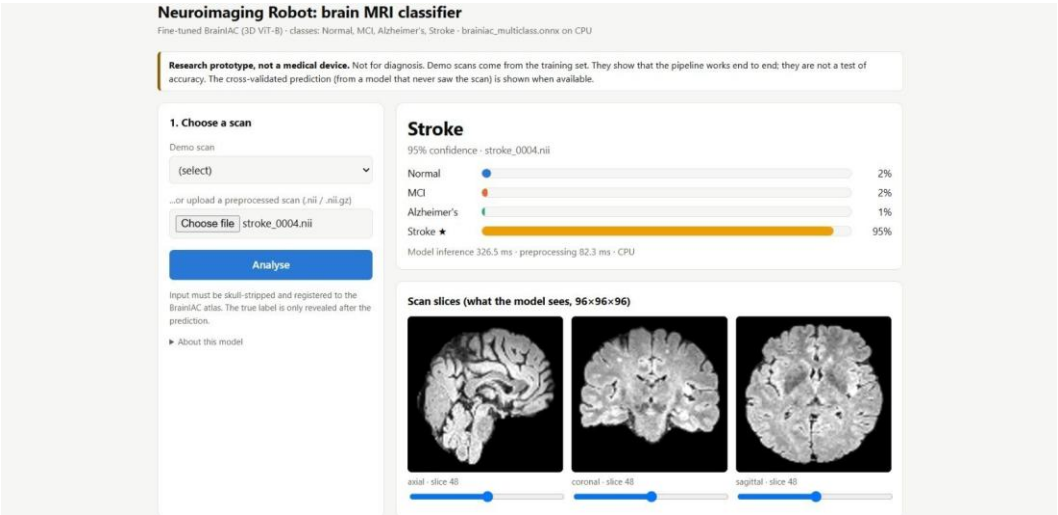


Figure 5.10. GUI example: Stroke scan output from the temporary interface

Each screenshot includes the three slice previews and the times reported by the application. The class bars add up to about 100%, as they should after softmax. That

normalisation does not make the scores calibrated, and the model can still put a high value on the wrong class. Whenever the GUI is demonstrated, the scores should be presented as model outputs and the research-use warning should stay visible.

5.9 TIMING AND INFERENCE CONSIDERATIONS

In the screenshots, inference takes between about 322 ms and 1,038 ms and preprocessing between about 82 ms and 410 ms on the CPU machine used. That suggests the temporary interface is quick enough for a demonstration. The figures are not benchmark results, though. The hardware details, warm-up conditions, other workloads on the machine and repeat-run statistics were not recorded.

The Jetson script is built to measure mean, median and p95 engine latency after a warm-up run. Those numbers should be kept apart from the time spent reading and preprocessing the NIfTI file. For a user, the number that matters is the full scan-to-result time, including inference and rendering. The Jetson test should also log memory use, GPU utilisation, power mode, thermal behaviour and software versions. We report no on-device latency here, because none of the files we have contain results from a real Jetson run.

5.10 LIMITATIONS AND THREATS TO VALIDITY

The main threats to the validity of these results are listed below.

- Small dataset: 60 volumes in total and only nine in the first holdout, so every metric carries wide uncertainty.
- Possible source confounding: Normal and Stroke appear to come from different datasets or sequences, and the package shortcut check reports near-perfect AUC from trivial image statistics.
- Possible overfitting: training accuracy rises while validation accuracy stays flat or falls, and validation loss climbs late in training.
- Different evaluation records: the nine-scan test and the packaged five-fold CV scores use different protocols and have not been reconciled from saved per-scan records.
- Possible class-index mismatch: the mapping in the training log differs from the class order in the Jetson package specification.
- Preprocessing dependency: the application expects registered, skull-stripped volumes, and the inference function shown does not support arbitrary raw clinical MRI.

- Incomplete device validation: there is no evidence yet of a TensorRT engine build, touchscreen use, on-device latency, or memory and power measurements.
- Clinical limits: no clinical cohort validation, calibration study, external test, reader comparison or regulatory assessment has been done.

5.11 SUMMARY

Taken together, the artifacts show a working software path from prepared NIfTI scans through fine-tuning, evaluation and ONNX export to a functioning GUI prototype. They also show why the results need careful reading. The first test missed every Alzheimer's case. The later package reports much stronger cross-validation metrics but also warns about likely source shortcuts. That is enough to justify more engineering work and a better-controlled evaluation. It is not enough to call the classifier ready for clinical use. The two things to fix first are the label mapping and the unexplained gap between the two evaluation records.

Chapter 6
Conclusion and Future Work

6.1 SUMMARY OF WORK

We built a BrainIAC-based 3D MRI classifier for four labels: Normal, MCI, Alzheimer's disease and Stroke. So far we have organised the four NIfTI class folders, fine-tuned a 3D ViT-B-based model, saved its training and evaluation outputs, exported it to ONNX, packaged an inference script with its specification and built a Gradio interface that shows class scores alongside axial, coronal and sagittal views.

We have also laid part of the path to edge deployment. The Jetson package defines the input tensor, the output logits, the class list, a reference output and the proposed TensorRT commands. These files make the next step concrete, but they do not show the model running on a Jetson. TensorRT output parity, touchscreen interaction and controlled hardware timing are still open. Table 6.1 summarises where each objective stands.

Table 6.1. Implementation status against project objectives

Objective	Status	Evidence / caveat
Organise four-class preprocessed data	Completed for current experiment	60 NIfTI files, 15 per class
Fine-tune BrainIAC-based 3D ViT-B	Implemented in notebook	Ten-epoch run and training logs provided
Evaluate class-wise performance	Partially completed	Initial held-out test and later CV metadata; run provenance to be reconciled
Export ONNX model	Completed as an artifact	ONNX graph and spec included in package
Build interactive GUI	Prototype completed	Temporary Gradio CPU GUI screenshots provided
Verify class-index order end to end	Pending	Training log mapping differs from Jetson spec mapping
Build TensorRT engine on Jetson	Pending / not evidenced	README provides commands; engine not included
Run GUI on touchscreen	Pending / not evidenced	No hardware screenshot or test log supplied
External clinical validation	Not completed; beyond current prototype	No clinical-use claim is made

6.2 KEY FINDINGS

- Adapting a foundation model made it possible to run a complete small-scale experiment with 3D MRI inputs and end up with a portable ONNX artifact.

- The nine-scan holdout result is modest, with 44.44% accuracy and a macro-F1 of 0.4464, and all three Alzheimer's scans were classified as MCI in that split.
- The later package's subject-grouped cross-validation reports better overall numbers, but its own shortcut analysis warns that some classes can be told apart by source or sequence instead of pathology.
- The GUI makes inference and scan viewing easy to access, and one of its examples shows a wrong prediction, which is a useful reminder to read the scores cautiously.
- Consistent class indices and matching preprocessing are hard requirements for deployment, not documentation details.

6.3 LIMITATIONS

The biggest limitation is the size and make-up of the dataset. Sixty scans split evenly across folders does not mean the classes are balanced in patient characteristics or acquisition conditions. The first test set is too small for stable estimates, and the model missed every Alzheimer's case in it. The later package reports better cross-validation metrics, but the compact archive lacks the fold-level files needed to reproduce them independently. The mismatch between the training and package class orders also has to be resolved before the labels shown by the exported model can be trusted.

The system is not yet a standalone edge product either. The GUI runs inference on a CPU, and the Jetson package contains scripts and instructions, not recorded device results. We cannot claim verified Jetson performance, a usable touchscreen interface or a finished robotic platform. Those remain planned deliverables, and they need testing on the actual hardware.

6.4 FUTURE WORK

First, settle on one canonical model version. Record its checkpoint, architecture, four-class mapping, ONNX metadata, dependency versions and file hashes. Then run the same labelled reference scans through PyTorch and ONNX Runtime and compare both the logits and the predicted labels. The GUI release and the TensorRT stage should wait until the mapping is confirmed for all four classes.

Second, make the evaluation reproducible. Save the subject identifiers, fold assignments, out-of-fold predictions, preprocessing configuration and evaluation code. Keep the nine-scan holdout as the record of the early experiment instead of replacing it with the later cross-validation numbers. Report per-class results and sample counts next to the macro averages, and give confidence intervals where the sample size allows. The

three-class within-source test should stay separate from the four-class experiment, since the Stroke data may carry source-specific cues.

Third, improve the dataset before making stronger claims. Where possible, use properly licensed scans from more than one source for each class, so that source and diagnosis are not tied together. Split by subject, make sure repeat scans cannot cross between folds, document the preprocessing history and hold back an external cohort for final testing if one is available.

Fourth, make training easier to reproduce. Explain every missing or unexpected checkpoint key, record all hyperparameters and use grouped validation to choose checkpoints. Treat frozen-backbone and full fine-tuning as an experiment to run, not an assumption. A supervised baseline and some preprocessing ablations would also show which parts of the current setup actually drive the results.

Fifth, test on the target Jetson. Record the JetPack, TensorRT and PyTorch versions, build the engine from the supplied ONNX graph and compare FP16 against FP32. After warm-up, time repeated runs and report the mean, median and p95. Log peak memory, GPU load, power mode, temperature and the total scan-to-result time, then repeat the checks with labelled scans from all four classes.

Sixth, run the application locally on the touchscreen. Check that file selection, buttons, sliders, score labels and warning text are readable and usable at the screen's native resolution. Test corrupted or missing files, volumes with no useful non-zero voxels, scans that skipped preprocessing, and application restarts. The setup should also be tested with networking switched off, and data should stay on the device unless sharing has been explicitly approved.

Only after the core pipeline is stable should the team add extras such as explainability maps, local logging, camera or microphone input, PACS access or report generation. These could be useful later. They should not hold up the basics: consistent preprocessing, correct class labels, matching inference outputs, stable operation on the device and a clear statement of what the model cannot do.

6.5 FINAL CONCLUSION

The project has grown well beyond a training notebook. It now includes model fine-tuning, evaluation artifacts, ONNX export and a GUI that displays results for a prepared MRI volume. The numbers also show where the approach falls short. The first holdout missed every Alzheimer's case, the later package reports better cross-validation results while flagging a likely source shortcut, and the class mapping differs between the training log and the deployment specification. These issues have to be resolved before anyone calls the classifier reliable. The practical next steps are to verify the

output mapping and PyTorch-to-ONNX agreement, and then to finish and measure the Jetson, TensorRT and touchscreen workflow. Until independent testing supports a stronger claim, the system remains an academic research prototype.

REFERENCES

[1] Tak, D., Garomsa, B. A., Zapaishchykova, A., Chaunzwa, T. L., Climent Pardo, J. C., Ye, Z., … Kann, B. H. (2026). A generalizable foundation model for analysis of human brain MRI. *Nature Neuroscience, 29*, 945–956. https://doi.org/10.1038/s41593-026-02202-6

[2] Marcus, D. S., Wang, T. H., Parker, J., Csernansky, J. G., Morris, J. C., & Buckner, R. L. (2007). Open Access Series of Imaging Studies (OASIS): Cross-sectional MRI data in young, middle aged, nondemented, and demented older adults. *Journal of Cognitive Neuroscience, 19*(9), 1498–1507. https://doi.org/10.1162/jocn.2007.19.9.1498

[3] Cox, R. W., Ashburner, J., Breman, H., Fissell, K., Haselgrove, C., Holmes, C. J., Lancaster, J. L., Rex, D. E., Smith, S. M., Woodward, J. B., & Strother, S. C. (2004). A (sort of) new image data format standard: NIfTI-1. *NeuroImage, 22*(Suppl. 1), e1440.

[4] Chen, T., Kornblith, S., Norouzi, M., & Hinton, G. (2020). A simple framework for contrastive learning of visual representations. In *Proceedings of the 37th International Conference on Machine Learning* (pp. 1597–1607). PMLR.

[5] AIM-KannLab. (n.d.). *BrainIAC: Brain Imaging Adaptive Core* [Source code repository]. GitHub. https://github.com/AIM-KannLab/BrainIAC

[6] Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., Uszkoreit, J., & Houlsby, N. (2021). An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations*. https://arxiv.org/abs/2010.11929

[7] Wang, C., Jiang, Y., Peng, Z., Li, C., Bang, C., Zhao, L., … Yuan, Y. (2026). *Towards a general-purpose foundation model for fMRI analysis* [Preprint]. arXiv. https://arxiv.org/abs/2506.11167

[8] MONAI Consortium. (2022). *MONAI: Medical Open Network for AI* [Computer software]. https://github.com/Project-MONAI/MONAI

[9] Open Neural Network Exchange. (n.d.). *ONNX: Open Neural Network Exchange*. https://onnx.ai/

[10] Project team. (2026). *Preprocessed four-class NIfTI scan collection (by_class.zip)* [Unpublished dataset archive].

[11] Project team. (2026). *Neuroimaging robot powered by brain foundation models: Preprocessing, fine-tuning notebook and result screenshots* [Kaggle notebook]. https://www.kaggle.com/code/maharghakundu/notebook0384ac332b

[12] Project team. (2026). *BrainIAC 4-class brain MRI classifier: Jetson inference package* [Unpublished project artifact archive].

APPENDICES

The appendices collect implementation details so the main chapters can concentrate on design choices and results. They record the supplied filenames, the model interface, the test procedure, metric definitions and the deployment checklist. They support reproducibility, but the original notebook remains the primary record, and nothing here is a new experiment.

APPENDIX 1: DATASET MANIFEST

The tables below list the 60 files found in by_class.zip, grouped by folder. The folder name is the only source of each label recorded here. The archive does not hold enough information to reconstruct patient identity, demographics, scanner settings or full diagnostic provenance. Those fields should be added to a controlled manifest before the next round of external testing.

A1.1 ALZHEIMER'S CLASS FILES

Table A1.1. Scan manifest for Alzheimer's class

Filename	Filename
OAS1_0028_MR1.nii.gz	OAS1_0031_MR1.nii.gz
OAS1_0053_MR1.nii.gz	OAS1_0056_MR1.nii.gz
OAS1_0067_MR1.nii.gz	OAS1_0137_MR1.nii.gz
OAS1_0185_MR1.nii.gz	OAS1_0223_MR1.nii.gz
OAS1_0269_MR1.nii.gz	OAS1_0316_MR1.nii.gz
OAS1_0351_MR1.nii.gz	OAS1_0388_MR1.nii.gz
OAS1_0399_MR1.nii.gz	OAS1_0405_MR1.nii.gz
OAS1_0425_MR1.nii.gz	

A1.2 MCI CLASS FILES

Table A1.2. Scan manifest for MCI class

Filename	Filename
OAS1_0022_MR1.nii.gz	OAS1_0023_MR1.nii.gz
OAS1_0039_MR1.nii.gz	OAS1_0046_MR1.nii.gz
OAS1_0066_MR1.nii.gz	OAS1_0094_MR1.nii.gz
OAS1_0123_MR1.nii.gz	OAS1_0124_MR1.nii.gz
OAS1_0205_MR1.nii.gz	OAS1_0272_MR1.nii.gz
OAS1_0288_MR1.nii.gz	OAS1_0307_MR1.nii.gz
OAS1_0418_MR1.nii.gz	OAS1_0440_MR1.nii.gz
OAS1_0451_MR1.nii.gz	

A1.3 NORMAL CLASS FILES

Table A1.3. Scan manifest for Normal class

Filename	Filename
OAS1_0069_MR1.nii.gz	OAS1_0085_MR1.nii.gz
OAS1_0110_MR1.nii.gz	OAS1_0139_MR1.nii.gz
OAS1_0146_MR1.nii.gz	OAS1_0165_MR1.nii.gz
OAS1_0203_MR1.nii.gz	OAS1_0204_MR1.nii.gz
OAS1_0208_MR1.nii.gz	OAS1_0254_MR1.nii.gz
OAS1_0280_MR1.nii.gz	OAS1_0332_MR1.nii.gz
OAS1_0345_MR1.nii.gz	OAS1_0363_MR1.nii.gz
OAS1_0443_MR1.nii.gz	

A1.4 STROKE CLASS FILES

Table A1.4. Scan manifest for Stroke class

Filename	Filename
stroke_0001.nii	stroke_0002.nii
stroke_0003.nii	stroke_0004.nii
stroke_0005.nii	stroke_0006.nii
stroke_0007.nii	stroke_0008.nii
stroke_0009.nii	stroke_0010.nii
stroke_0011.nii	stroke_0012.nii
stroke_0013.nii	stroke_0014.nii
stroke_0015.nii	

APPENDIX 2: MODEL INPUT/OUTPUT CONTRACT

The preprocessing and model interface should be handled as a versioned contract. If the tensor shape or the class order changes, the metadata and the tests have to change with it. Otherwise inference can run without any error while the GUI attaches an output to the wrong label.

Table A2.1. Input and output tensor specification

Field	Value recorded in Jetson package
Task	Four-class brain MRI classification
Input tensor name	mri
Input tensor shape	[1, 1, 96, 96, 96]
Input type	float32
Output tensor name	logits
Output tensor shape	[1, 4]
Class array	normal, mci, alzheimers, stroke
Post-processing	Softmax across four logits; argmax for predicted class
Resize	Trilinear to 96 × 96 × 96, align_corners=False
Intensity normalisation	Z-score on non-zero voxels; zero voxels remain zero
Upstream preparation	N4 correction, 1 mm resampling, rigid registration to template, HD-BET skull stripping
Reference files	spec.json; expected.json; sample_input.bin; sample_scan.nii.gz

Release check: the training screenshot shows a different class-index mapping from the Jetson specification. The application should not be released until every index has been checked against the notebook, the saved checkpoint, the ONNX and TensorRT outputs, the GUI labels and labelled reference tests.

APPENDIX 3: INFERENCE TEST PROCEDURE

The steps below come from the supplied Jetson README and are meant as a test plan. Paths and commands should be adapted to the installed device. Listing them here records the planned procedure; none of them has yet been run successfully on the target hardware.

```
# 1. Record device and framework versions
cat /etc/nv_tegra_release
python3 -c "import tensorrt, torch; print(tensorrt.__version__, \
            torch.__version__, torch.cuda.is_available())"

# 2. Set the power mode and clocks (only as permitted for the device)
sudo nvpmodel -m 0
sudo jetson_clocks

# 3. Build a device-specific TensorRT engine from the ONNX graph
/usr/src/tensorrt/bin/trtexec --onnx=brainiac_multiclass.onnx \
    --saveEngine=brainiac_fp16.engine --fp16

# 4. Run repeated engine inference and dump a reference output
/usr/src/tensorrt/bin/trtexec --loadEngine=brainiac_fp16.engine \
    --loadInputs=mri:sample_input.bin --dumpOutput --iterations=50

# 5. Run the NIfTI inference script and compare with the reference JSON
python3 jetson_infer.py --engine brainiac_fp16.engine \
    --scan sample_scan.nii.gz --spec spec.json --expected expected.json

# 6. In another terminal, collect performance telemetry
tegrastats
```

A3.1 REQUIRED TESTS

- Run model parity tests on a set of model-ready scans with known labels from all four classes.
- Compare PyTorch and ONNX logits, and record the numerical difference, the score difference and whether the predicted classes agree.
- Compare ONNX Runtime FP32, TensorRT FP32 (if available) and TensorRT FP16 outputs.
- Benchmark the model after warm-up and record the mean, median, p95, standard deviation, number of runs and end-to-end latency.
- Measure peak system RAM, GPU utilisation, thermal state, power mode and application memory on the actual Jetson.
- Test missing files, corrupted NIfTI files, non-finite values, empty or all-zero volumes, wrong orientation and scans that were never registered or skull-stripped.

- Run the GUI on the touchscreen at its target resolution and check that text, buttons, sliders, warnings and score labels stay readable.

A3.2 DEVICE VALIDATION RECORD SHEET

This sheet should be filled in only after running the application on the actual Jetson and touchscreen. The fields are left blank on purpose. Deployment results belong here once they are measured, not estimated from the intended setup or from a desktop screenshot.

Table A3.1. Device validation record sheet

Field	Record during validation
Test date / operator	To be completed
Jetson SKU / RAM / storage	To be completed
JetPack / CUDA / TensorRT versions	To be completed
Model filename / SHA-256	To be completed
ONNX input/output specification and class order	To be completed
Reference scan ID and verified label	To be completed
Source-framework logits / predicted label	To be completed
ONNX Runtime logits / predicted label	To be completed
TensorRT FP32 logits / predicted label	To be completed
TensorRT FP16 logits / predicted label	To be completed
Maximum absolute logit difference / class agreement	To be completed
Warm-up count / timed iterations	To be completed
Mean / median / p95 engine latency	To be completed
End-to-end scan-to-result latency	To be completed
Peak RAM / GPU use / power mode / temperature	To be completed
Touchscreen resolution / UI checks	To be completed
Pass/fail / failure notes / log filename	To be completed

APPENDIX 4: METRIC DEFINITIONS

Table A4.1. Metric definitions

Metric	Formula / description
Accuracy	(TP + TN) / (TP + TN + FP + FN) for binary tasks; total correct / total samples for multiclass
Precision	TP / (TP + FP)
Recall / sensitivity	TP / (TP + FN)
Specificity	TN / (TN + FP)
F1-score	2 × (precision × recall) / (precision + recall)
Macro average	Unweighted mean of the class-wise metric values
Weighted average	Class-wise metric weighted by number of true instances
Balanced accuracy	Arithmetic mean of class recalls
ROC-AUC	Area under the ROC curve; in multiclass one-vs-rest, computed for each class against the rest
Average precision	Summary of the precision-recall curve based on ranked model scores
Mean latency	Arithmetic mean of measured inference durations after warm-up
P95 latency	95th percentile of measured inference durations

The first test set has three Alzheimer's examples and two of each other class. A single prediction therefore shifts Alzheimer's recall by 33.33 percentage points and shifts recall for MCI, Normal or Stroke by 50 percentage points. That is why class support should always appear next to the metrics, and why the curves from the first run need cautious reading.

APPENDIX 5: JETSON AND TOUCHSCREEN CHECKLIST

Table A5.1. Jetson and touchscreen validation checklist

Item	Acceptance criterion	Status from supplied evidence
Confirm target device SKU	Record exact Jetson model, RAM, storage and power mode	Pending
Record software stack	Record JetPack, CUDA, TensorRT, Python and ONNX tooling versions	Pending on target
Copy model artifacts	ONNX, spec, expected output, reference input and preprocessing template are present	Package prepared
Verify class order	All four labels match checkpoint, ONNX and TensorRT logits	Open issue
Numerical parity	Outputs agree within documented tolerance and keep class agreement	Pending
Build TensorRT engine	Engine builds and loads on the target device	Instructions provided; run not evidenced
Benchmark model	Mean, median and p95 engine latency over repeated warm runs	Pending
Benchmark full pipeline	Includes loading, preprocessing, inference and UI update	Pending
Observe device health	RAM, GPU, power, temperature and throttling recorded	Pending
Run GUI locally	Starts without cloud dependence and loads the model from local storage	Pending
Touchscreen usability	All controls and warnings readable at native resolution	Pending
Input validation	Rejects unsupported, unregistered or malformed scans with a clear message	Needs testing
Offline test	Application works with networking disabled	Pending
Safety notice	Visible statement: research prototype, not for diagnosis	Already visible in current GUI
External validation	Independent subject-level dataset with source-aware splits	Future work

APPENDIX 6: ARTIFACT INVENTORY

Table A6.1. Deliverable artifact inventory

Artifact	Included where / evidence	Purpose
by_class.zip	Uploaded dataset archive	60 preprocessed NIfTI inputs organised by four classes
Kaggle notebook	Notebook URL in report	Preprocessing and fine-tuning workflow
best_brainiac_finetuned.pth	Notebook output archive listing (screenshot)	PyTorch checkpoint; not included in compact Jetson ZIP
brainiac_4class_finetuned.onnx	Notebook output archive listing (screenshot)	ONNX export from training run
brainiac_multiclass.onnx	jetson_package_4class.zip	ONNX model packaged for deployment
spec.json	jetson_package_4class.zip	Tensor I/O, classes, CV metrics, normalisation details
expected.json	jetson_package_4class.zip	Reference scan label, logits and softmax values
sample_input.bin	jetson_package_4class.zip	Binary reference input for engine test
sample_scan.nii.gz	jetson_package_4class.zip	Example NIfTI input for inference script
jetson_infer.py	jetson_package_4class.zip	TensorRT inference and timing script
README_FOR_JETSON.md	jetson_package_4class.zip	Device setup and build/benchmark instructions
Gradio screenshots	Uploaded report evidence	GUI prediction, visualisation and timing examples
Evaluation screenshots	Uploaded report evidence	Training curves, held-out metrics, confusion matrix, ROC and PR curves

APPENDIX 7: FULL TRAINING HISTORY

This table transcribes the ten-epoch training history from the uploaded screenshot. It belongs to the first training run and should be kept apart from the cross-validation metrics reported later in the Jetson package.

Table A7.1. Training and validation history by epoch

Epoch	Train loss	Train acc.	Train F1	Val. loss	Val. acc.	Val. F1
1	1.4232	0.381	0.317	1.2941	0.444	0.361
2	1.4068	0.452	0.392	1.2946	0.444	0.325
3	1.0094	0.429	0.323	0.9219	0.444	0.361
4	0.8446	0.476	0.407	0.8794	0.444	0.361
5	0.8450	0.452	0.382	0.8643	0.444	0.421
6	0.8298	0.524	0.381	0.9413	0.444	0.361
7	0.7923	0.548	0.430	0.8736	0.444	0.421
8	0.7468	0.643	0.559	0.9197	0.444	0.375
9	0.6754	0.714	0.622	1.0092	0.444	0.375
10	0.6204	0.738	0.642	1.1224	0.333	0.321

APPENDIX 8: FINAL REPORTING CHECKLIST

- Every reported metric states the dataset size, split protocol, class support, model version and where the value came from.
- The first held-out result stays clearly separate from the package-reported cross-validation metrics.
- No metric from the final model, refit on all 60 scans, is reported as independent test performance.
- The class-order mismatch is resolved and regression tests cover all four outputs before final submission.
- Source shortcut checks and within-source disease metrics stay visible in results and presentations.
- The report states that the tool is a research prototype and not for diagnosis.
- The Jetson, TensorRT and touchscreen work is marked complete only after test logs and photographs from the actual setup have been saved.